

บทที่ 6

ไคเร็กเอ็กรกราฟิก (DirectX Graphics)

6.1 แนะนำไคเรทรีดี (Introduction to Direct3D)

ไคเรทรีดี คือ แอปพลิเคชันโปรแกรมมิ่งอินเทอร์เฟซระดับล่าง สำหรับนักพัฒนาโปรแกรมที่ต้องการสร้างเกมหรือโปรแกรมมัลติมีเดียประสิทธิภาพสูงบนระบบปฏิบัติการวินโดวส์ ซึ่งการใช้ไคเรทรีดีทำให้โปรแกรมสามารถติดต่อกับฮาร์ดแวร์เร่งความเร็วได้ในระดับล่าง (โดยตรง)

ไคเรทรีดี คือ ซอฟต์แวร์อินเทอร์เฟซ (software interface) ซึ่งจัดการการเข้าถึงอุปกรณ์แสดงผล (display device) โดยยังคงรักษาความเข้ากันได้กับกราฟิกดีไวซ์อินเทอร์เฟซ (Graphics Device Interface : GDI) และจัดการให้มีการใช้งานในแบบที่ไม่ขึ้นกับชนิดของอุปกรณ์ (device independent) และมีความสามารถในการใช้คุณสมบัติพิเศษที่อยู่ในฮาร์ดแวร์ได้

6.2 คุณสมบัติของ ไคเรทรีดี

- สนับสนุนการใช้งาน depth buffer (z-buffer หรือ w-buffer)
- สามารถเรนเดอร์ภาพได้ทั้งในแบบ Flat shading และ Gouraud shading
- สามารถใช้แหล่งแสงได้หลายแหล่งและหลายชนิด
- สนับสนุนการใช้แมตทีเรียล (material) และเทกเจอร์ (texture) รวมทั้ง การทำมิพแมพ (mipmap)
- มีซอฟต์แวร์อีมูเลชันไดเวอร์ (software emulation driver) ที่มีประสิทธิภาพสูง
- ไม่ขึ้นอยู่กับชนิดของฮาร์ดแวร์
- สนับสนุนคำสั่งพิเศษที่มีอยู่ในซีพียู เช่น MMX, SSE และ 3DNow!
- สนับสนุน Hardware Abstraction Layer (HAL)
- สนับสนุนการทำ page flipping ด้วย back buffer จำนวนมากในโปรแกรมที่แสดงผลแบบเต็มจอภาพ (fullscreen)
- สนับสนุนการตัด (clipping) ในโปรแกรมที่ทำงานในวินโดวส์และแบบเต็มจอภาพ
- สามารถใช้งาน image-stretching hardware ได้
- สามารถใช้งานเมมโมรีของอุปกรณ์แสดงผลทั้งส่วนที่เป็นมาตรฐานและส่วนที่ขยายเพิ่มเติมได้
- คุณสมบัติอื่นๆ เช่น การใช้งานฮาร์ดแวร์แต่เพียงผู้เดียว (exclusive hardware access) และการเปลี่ยนความละเอียดในการแสดงผล (resolution switching)

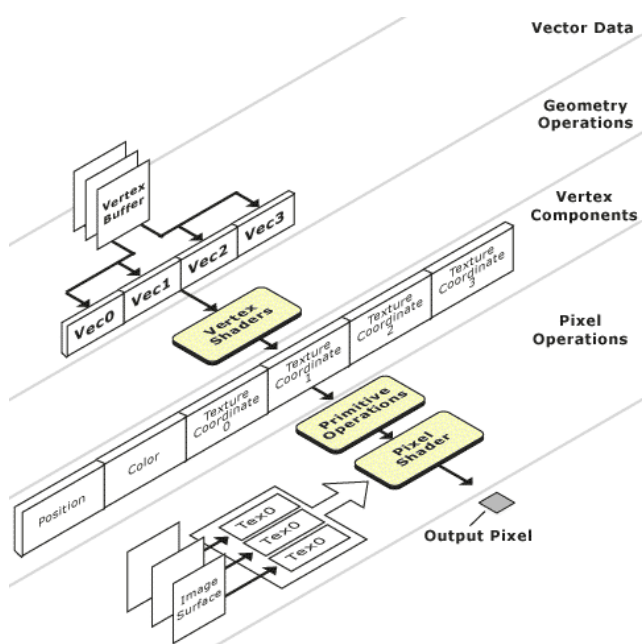
ด้วยคุณสมบัติเหล่านี้การสร้างโปรแกรมด้วยไคเรทรีดีจึงมีประสิทธิภาพสูงกว่าการใช้ GDI ของวินโดวส์และการเขียนโปรแกรมบนระบบปฏิบัติการคอส

การจัดการสิ่งต่างๆ ในไคเรทรีดีมีพื้นฐานอยู่บนทฤษฎีของเวอร์เท็กซ์, โพลีกอนและคำสั่งต่างๆ ที่ควบคุมข้อมูลเหล่านี้ ซึ่งสามารถเข้าถึงไปป์ไลน์ของกราฟิก 3 มิติในส่วนของ การเปลี่ยนแปลงตำแหน่ง (transformation), การให้แสง (lighting) และ การลงจุดเพื่อสร้างภาพ (rasterization) ได้ทันที ซึ่งถ้าฮาร์ดแวร์ที่มีอยู่ในเครื่องไม่สนับสนุนการเร่งความเร็วในส่วนใดของไปป์ไลน์ก็ตาม ไคเรทรีดีจะใช้ซอฟต์แวร์ที่มี

ประสิทธิภาพสูงในการทำงานในส่วนนั้นแทน (แต่การทำงานของซอฟต์แวร์ก็ยังมีประสิทธิภาพต่ำกว่าการใช้ฮาร์ดแวร์อยู่ดี)

6.3 สถาปัตยกรรมของไดเรกทรีดี (Direct3D Architecture)

ในไดเรกทรีดีมี 2 ส่วนที่สามารถโปรแกรมได้คือ เวอร์เท็กซ์เชดเดอร์ (vertex shader) และ พิกเซลเชดเดอร์ (pixel shader) ซึ่งเวอร์เท็กซ์เชดเดอร์เป็นส่วนที่จัดการการสร้างและกระบวนการต่างๆ บนเวอร์เท็กซ์ ส่วนพิกเซลเชดเดอร์ทำงานหลังจากมีการประมวลผลข้อมูล ออกมาเป็นตำแหน่งของพิกเซลและค่าสีของพิกเซลนั้นๆ แล้ว ดังภาพ



รูปที่ 6-1 โครงสร้างการทำงานของไดเรกทรีดี

จะเห็นว่าการทำงานของเวอร์เท็กซ์เชดเดอร์จะรับข้อมูลมาจากบัฟเฟอร์ ซึ่งโดยปกติแล้วจะเป็นเวอร์เท็กซ์บัฟเฟอร์ (vertex buffer) ที่เก็บข้อมูลเวอร์เท็กซ์ และนำมาผลกระบวนการต่างๆ ซึ่งสามารถกำหนดกระบวนการที่จะทำได้โดยใช้ DirectX vertex shader assembler

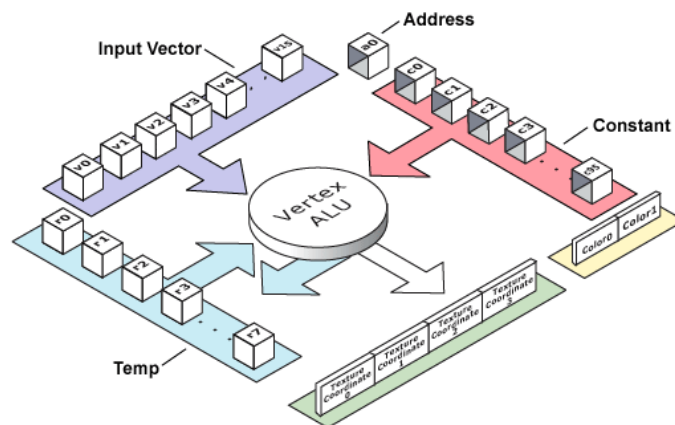
หลังจากการข้อมูลผ่านการประมวลผลในเวอร์เท็กซ์เชดเดอร์แล้วจะต้องเรียกฟังก์ชันในการวาดข้อมูลเหล่านี้ เช่น DrawPrimitive เป็นต้น ซึ่งเมื่อเรียกฟังก์ชันเหล่านี้แล้วจุดแต่ละจุดที่ได้จากฟังก์ชันก็จะส่งเข้าสู่พิกเซลเชดเดอร์ ข้อมูลที่ส่งให้กับพิกเซลเชดเดอร์ประกอบด้วยตำแหน่งบนจอภาพ, สี, เทกเจอร์และ กลุ่มอันดับแสดงตำแหน่งของเทกเจอร์ พิกเซลเชดเดอร์จะประมวลผลตามคำสั่งที่กำหนดโดยใช้ DirectX pixel assembler

นอกจาก ข้อมูลที่ได้รับจากฟังก์ชันในการวาดข้อมูล เช่น DrawPrimitive แล้ว พิกเซลเชดเดอร์ยังต้องการข้อมูลของเทกเจอร์ในรูปแบบของเทกเจอร์เซอร์เฟส (texture surface) อีกด้วยซึ่งจะให้ร่วมกับข้อมูลกลุ่มอันดับ

แสดงตำแหน่งของเทกเจอร์ ในการกำหนดสีที่จะปรากฏ แล้วจึงส่งค่าพิกเซล (pixel) ไปยังเฟรมบัพเฟอร์ (frame buffer) เพื่อแสดงผลออกจอภาพต่อไป

6.3.1 สถาปัตยกรรมของเวอร์เท็กซ์เชดเดอร์ (Vertex Shader Architecture)

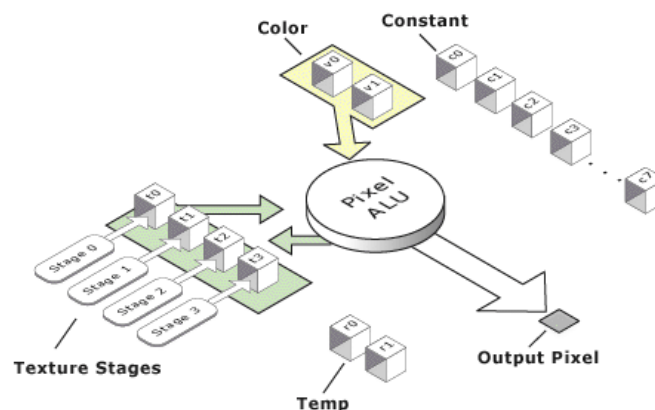
เวอร์เท็กซ์เชดเดอร์รับข้อมูลมาจากเวอร์เท็กซ์ดาต้าสตรีม (vertex data stream) และมีข้อมูลค่าคงที่ที่มีความจำเป็นในการคำนวณ ซึ่งสามารถอ่านเข้ามาได้โดยใช้เวอร์เท็กซ์เชดเดอร์ดีคลาเรเตอร์ (vertex shader declarator) ดังแผนภาพ



รูปที่ 6-2 สถาปัตยกรรมของเวอร์เท็กซ์เชดเดอร์

6.3.2 สถาปัตยกรรมของพิกเซลเชดเดอร์ (Pixel Shader Architecture)

พิกเซลเชดเดอร์จะต้องอ่านข้อมูลที่เป็น clip space vertex (homogeneous coordinate) ค่าของสีดิฟฟิวส์ (diffuse) และสเปกคิวลาร์ (specular) มาจากค่าสีในรีจิสเตอร์สี (color register) v0 และ v1 ค่าคู้ดับแสดงตำแหน่งของเทกเจอร์และเทกเจอร์ที่ใช้มาจากค่าในรีจิสเตอร์เทกเจอร์ (texture register) t0, t1, t2 และ t3 จากข้อมูลในขั้นตอนการกำหนดเทกเจอร์ (texture setup stage) และข้อมูลในเวอร์เท็กซ์ ดังภาพ



รูปที่ 6-3 สถาปัตยกรรมของพิกเซลเชดเดอร์

6.4 เรนเดอร์ไปป์ไลน์ในไดเรกทรีดี (Direct3D Rendering Pipeline)

เรนเดอร์ไปป์ไลน์ (rendering pipeline) คือ ชุดของขั้นตอนในการประมวลผลซึ่งกำหนดการกระทำของไปป์ไลน์ในการเปลี่ยนแปลงตำแหน่งและให้แสงของเวอร์เท็กซ์ (vertex transform and lighting pipeline) และ ไปป์ไลน์ในการเฉลี่ยค่าของพิกเซลและเทกเจอร์ (pixel and texture blending pipeline)

ในไดเรกทรีดีมีเรนเดอร์ไปป์ไลน์ 2 แบบ คือ การประมวลผลฟังก์ชันแบบตายตัว (fix function vertex and pixel processing) และการประมวลผลเวอร์เท็กซ์และพิกเซลแบบโปรแกรมได้ (Programmable Vertex and Pixel Processing) ซึ่งมีรายละเอียด ดังนี้

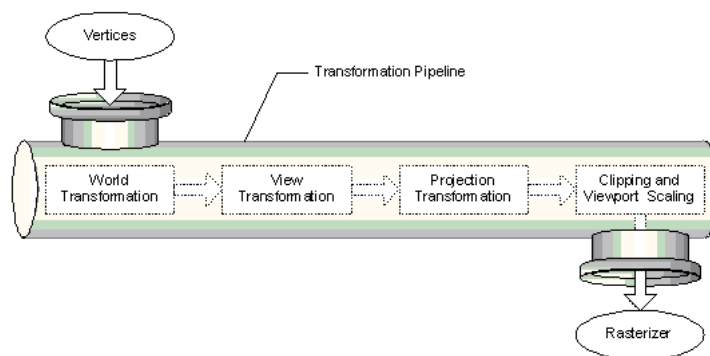
6.4.1 การประมวลผลฟังก์ชันแบบตายตัว (Fix Function Vertex and Pixel Processing)

ในไดเรกทรีดีการประมวลผลฟังก์ชันแบบตายตัวจะใช้ไปป์ไลน์ที่เรียกว่า Transformation Pipeline หรือ Geometry Pipeline ซึ่งมีส่วนประกอบ ดังนี้

Transformation engine

ทำหน้าที่ในการกำหนดตำแหน่งของโมเดลและผู้ดู (viewer) ในโลกของไดเรกทรีดี, ฉายเวอร์เท็กซ์ (project vertex) สำหรับการแสดงผลบนจอภาพ และตัด (clip) เวอร์เท็กซ์ส่วนที่มองไม่เห็นทิ้ง นอกจากนี้ยังทำงานในส่วนของการฉายด้วยโดยการคำนวณค่าของดิฟฟิวส์ และสเปกคิวลาร์ในแต่ละเวอร์เท็กซ์

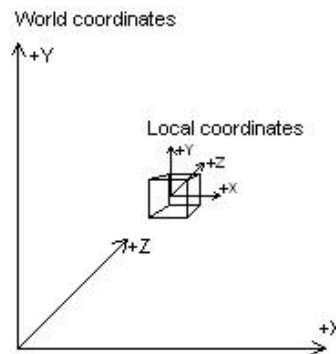
Transformation pipeline รับเวอร์เท็กซ์เข้ามาแล้วประมวลผลโดยใช้ world, view และ projection transformation ซึ่งอยู่ในรูปของเมตริกซ์ หลังจากนั้นข้อมูลเวอร์เท็กซ์จะถูกตัดให้เหลือแต่ส่วนที่มองเห็น แล้วจึงส่งไปยังส่วนของการลงจุดสีต่อไป ดังภาพ



รูปที่ 6-4 เรนเดอร์ไปป์ไลน์ในไดเรกทรีดี

World transformation

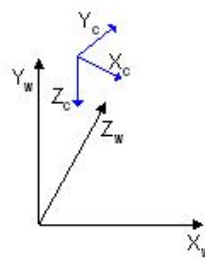
คือ การเปลี่ยนแปลงตำแหน่งของเวอร์เท็กซ์จากเดิมที่มีความสัมพันธ์กับตัวเองเท่านั้น (model space) ให้เปลี่ยนมาอ้างอิงตำแหน่งใหม่ซึ่งใช้อ้างอิงกับวัตถุทั้งหมดที่จะปรากฏในฉาก (world space) ดังภาพ



รูปที่ 6-5 รูปแสดงความสัมพันธ์ระหว่างตำแหน่งของโมเดลใน world coordinates

View transformation

เป็นการกำหนดตำแหน่งของผู้ดู (viewer) หรือกล้อง (camera) ใน world space



รูปที่ 6-6 รูปแสดงความสัมพันธ์ระหว่างตำแหน่งของโมเดลใน view coordinates

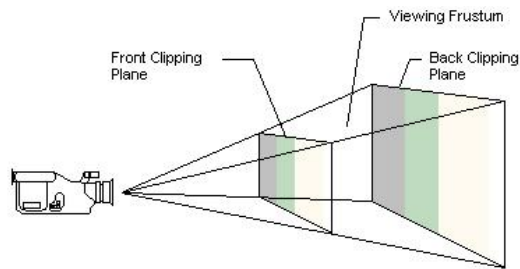
Projection transformation

เป็นการกำหนดค่าภายในของกล้องในทำนองเดียวกับการเลือกเลนส์ที่จะใช้ในกล้องถ่ายภาพ ซึ่งเป็นส่วนที่มีความซับซ้อนมากที่สุดในการ transformation ทั้งสามแบบ ประกอบด้วยหัวข้อต่อไปนี้

View Frustum

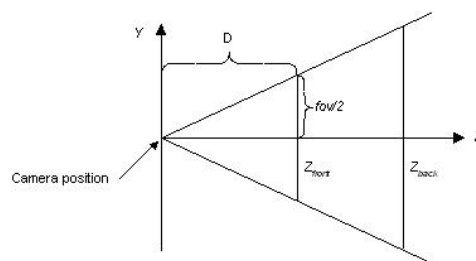
View frustum คือ บริเวณ 3 มิติที่จะปรากฏโดยมีความสัมพันธ์กับตำแหน่งและมุมมองของกล้อง (viewport's camera) รูปร่างของบริเวณ 3 มิติที่เกิดขึ้นจะเป็นตัวกำหนดว่าโมเดลจะโปรเจกต์บนจอภาพในลักษณะใด ซึ่งลักษณะการโปรเจกต์ที่ใช้ทั่วไป คือ แบบเพอร์สเปกทิฟ (perspective) ซึ่งวัตถุที่อยู่ใกล้จะดูมีขนาดใหญ่กว่าวัตถุที่อยู่ห่างออกไป

View frustum มีรูปร่างเหมือนปริมาตรซึ่งมีตำแหน่งของกล้องเป็นจุดยอด บริเวณ 3 มิติจะเกิดขึ้นจากการตัดปริมาตรนี้ด้วย ระนาบตัดด้านหน้า (front clipping plane) และระนาบตัดด้านหลัง (back clipping plane) และวัตถุจะมองเห็นก็ต่อเมื่อมันอยู่ในบริเวณนี้เท่านั้น ดังแผนภาพ



รูปที่ 6-7 รูปแสดง view frustum

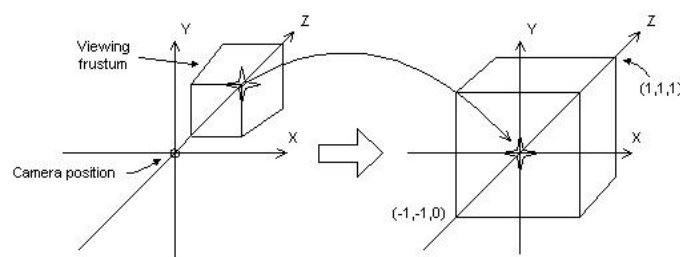
view Frustum มีพารามิเตอร์ที่สำคัญ คือ fov (field of view) และค่าระยะทางของระนาบตัดทั้งด้านหน้าและด้านหลัง ซึ่งกำหนดอยู่ในแกน z ดังแผนภาพ



รูปที่ 6-8 รูปแสดงการคำนวณค่า fov

Projection matrix

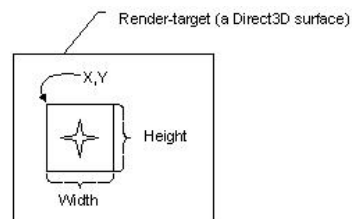
Projection matrix ใช้ในการปรับขนาดของ view frustum เนื่องจากบริเวณที่วัตถุสามารถมองเห็นได้จะต้องถูกปรับให้เป็นรูปสี่เหลี่ยมลูกบาศก์จึงจะสามารถประมวลผลต่อไปได้ ดังภาพ



รูปที่ 6-9 รูปแสดงวิธีการ projection

Viewport and clipping

Viewport คือ สี่เหลี่ยมที่โปรเจกต์ยังจากใน 3 มิติ ซึ่งใน ไคเรกทรีดี จะอยู่ในรูปของคู่อันดับใน Direct3D Surface ซึ่งระบบใช้เป็นบริเวณที่เรนเดอร์ภาพ ดังภาพ



รูปที่ 6-10 รูปแสดงวิวพอร์ตและวิธีการตัด

การลงจุด (Rasterization)

หลังจากผ่าน transformation pipeline แล้วค่าเวอร์เท็กซ์จะถูกแปลง, ตัด และย่อขยายจนมีขนาดพอดีกับวิวพอร์ต (viewport) บนเซอร์เฟสที่ต้องการเรนเดอร์ (render-target) คือ พร้อมที่จะส่งไปยังส่วนของการลงจุดสีเพื่อแสดงผลบนจอภาพ แต่เนื่องจากในขณะนี้ข้อมูลเวอร์เท็กซ์ยังคงอยู่แบบโฮโมจีเนียส (homogeneous) อยู่ และตัวลงจุดสียังต้องการข้อมูลของ reciprocal-of-homogeneous-w (RHW) เพิ่มเติมอีกด้วย ซึ่ง ไคเรกทรีดี จะทำการแปลงค่าตำแหน่งในแกน x, แกน y และแกน z ไปเป็นข้อมูลที่ไม่โฮโมจีเนียสโดยหารด้วยค่า w และหาค่า RHW จากส่วนกลับของค่า w ดังสมการ

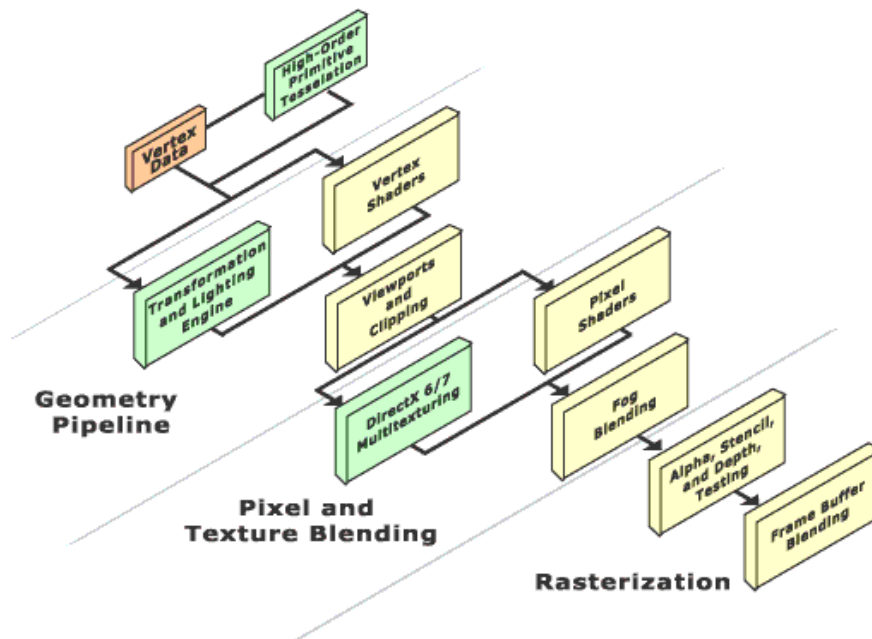
$$\begin{aligned}X_s &= x/w \\ y_s &= y/w \\ z_s &= z/w \\ RHW &= 1/w\end{aligned}$$

เมื่อได้ค่าที่ต้องการแล้วจึงส่งเข้าสู่ตัวลงจุดสี (rasterizer) ซึ่งจะใช้ค่าในแกน x และแกน y ในการกำหนดว่าเป็นจุดใดบนจอภาพ, และใช้ข้อมูลในแกน z เพื่อใช้วัดความลึกเพื่อเลือกว่าจุดสีใดที่จะได้แสดงผล โดยใช้ Z-Buffer ส่วนค่า RHW ใช้ในการคำนวณหมอก (fog), การทำปะเทกเจอร์ (texture mapping) และ ใช้ในการวัดความลึกเพื่อเลือกว่าจุดสีใดที่จะได้แสดงผลโดยใช้ W-Buffer

6.4.2 การประมวลผลเวอร์เท็กซ์และพิกเซลแบบโปรแกรมได้

(Programmable Vertex and Pixel Processing)

นอกจากไปป์ไลน์ในการประมวลผลเดิมที่มีอยู่ในเวอร์ชันเก่าๆ แล้วใน ไดรেকทรีดี เวอร์ชัน 8 ยังได้เพิ่มไปป์ไลน์ที่สามารถโปรแกรมได้อีกด้วย โดยมีส่วนที่เพิ่มเข้ามาดังภาพ



รูปที่ 6-11 รูปแสดงขั้นตอนในการประมวลผลเวอร์เท็กซ์และพิกเซล

ส่วนแรกที่เพิ่มเข้ามาคือ high-order primitive module ซึ่งทำหน้าที่ในการเทสเซลเลต (tessellate) high-order primitive เช่น bezier และ B-spline ส่วนเวอร์เท็กซ์เชดเดอร์ (vertex shader) เป็นโมดูลที่สามารถโปรแกรมได้ใช้ในการประมวลผลทางจีโอเมตริก (geometric) กับข้อมูลเวอร์เท็กซ์ ซึ่งมีความสามารถในการประมวลผลได้หลากหลายรวมทั้งมีความสามารถในการประมวลผลในแบบมาตรฐานเดิมได้ ส่วนพิกเซลเชดเดอร์ (pixel shader) เป็นโมดูลที่สามารถโปรแกรมได้เช่นเดียวกัน มีหน้าที่ในการควบคุมสีและการผสมสีในส่วนของความโปร่งใส (alpha channel) และในกระบวนการเกี่ยวกับคู้่อันดับกำหนดตำแหน่งของเทกเจอร์ (texture coordinate)

6.5 วัตถุไดเร็กทรีดี (Direct3D Object)

ไดเร็กทรีดี ถูกสร้างขึ้นโดยใช้หลักการของ COM object และอินเทอร์เฟส ดังนั้น โปรแกรมที่เขียนขึ้นในภาษาซีและซีพลัสพลัส (C/C++) จึงสามารถใช้งานอินเทอร์เฟสและออปเจกต์ได้โดยตรง ต่างจากการใช้ภาษาเบสิก (basic) หรือภาษาอื่นๆ ที่ต้องทำงานผ่านชั้นของโค้ดที่ทำหน้าที่ในการแปลงข้อมูลในการติดต่อกับ DirectX runtime อีกทอดหนึ่ง

วัตถุไดเรกทรีดีเป็นวัตถุแรกที่ต้องสร้างขึ้นถ้าต้องการจะใช้งานไดเรกทรีดี และเป็นวัตถุสุดท้ายที่จะต้องปลดปล่อย (release) หลังจากการใช้งาน เราสามารถตรวจสอบความสามารถของฮาร์ดแวร์และอุปกรณ์อื่นๆ ที่มีอยู่ได้โดยการส่งงานผ่านทางวัตถุไดเรกทรีดี ซึ่งจะทำให้เราสามารถตรวจสอบอุปกรณ์ได้โดยไม่ต้องสร้างมันขึ้นมา

การสร้างวัตถุไดเรกทรีดี

ขั้นตอนแรก คือการรับค่าพอยเตอร์ที่ชี้ไปยังไดเรกทรีดีอินเตอร์เฟซ เพื่อใช้ในการเข้าถึงการทำงานต่างๆ ของไดเรกทรีดี โดยสามารถทำได้ดังนี้

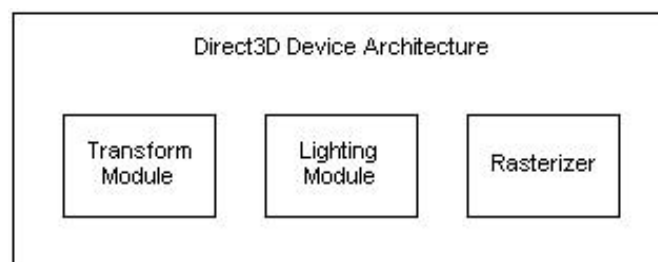
```
LPDIRECT3D8      g_pD3D = NULL;
if ( NULL == (g_pD3D = Direct3DCreate8(D3D_SDK_VERSION)))
    return E_FAIL;
```

เมื่อสร้างดีไวซ์ขึ้นมาแล้วสามารถใช้คำสั่งในการหาพอยเตอร์ที่ชี้ไปยัง ไดเรกทรีดีอินเตอร์เฟซที่ใช้สร้างดีไวซ์ได้โดยใช้เมทอด (method) IDirect3DDevice8::GetDirect3D

6.6 ดีไวซ์ในไดเรกทรีดี (Direct3D Device)

ดีไวซ์ในไดเรกทรีดี คือ องค์ประกอบที่ใช้ในการเรนเดอร์ ซึ่งได้เก็บรักษาสถานะในการเรนเดอร์เอาไว้ (render state) นอกจากนั้นยังทำหน้าที่ในกระบวนการเปลี่ยนแปลงตำแหน่งและให้แสง (transformation and lighting operation) และการลงจุด (rasterize) ของภาพ อีกด้วย

โครงสร้างของดีไวซ์ ดังภาพ



รูปที่ 6-12 รูปแสดงสถาปัตยกรรมการทำงานของดีไวซ์

ชนิดของดีไวซ์

HAL Device

Hardware Abstraction Layer (HAL) device คือ ฮาร์ดแวร์ซึ่งมีความสามารถในการเร่งความเร็วในการลงจุดดี นอกจากนั้นยังอาจมีความสามารถในการเร่งความเร็วในการประมวลผลเวอร์เท็กซ์ (vertex processing) อีกด้วย นั่นคือ HAL device มีการสร้างโมดูลในบางส่วนหรือทั้งหมดของเรนเดอร์ไปป์ไลน์ไว้ในตัวฮาร์ดแวร์ จึงมีประสิทธิภาพในการทำงานสูง แต่ HAL device ไม่สามารถเรนเดอร์บนเซอร์เฟสแบบ 8 บิตได้

การสร้างดีไวซ์แบบ HAL สามารถทำได้โดยเรียกเมทอด `IDirect3D8::CreateDevice` และกำหนดค่า `D3DDEVTYPE_HAL` เป็นชนิดของดีไวซ์

Reference Device

Reference device สนับสนุนการทำงานของ ไดรเรทรีดี ในทุกรูปแบบ เนื่องจากมันได้รับการออกแบบให้ทำงานอย่างถูกต้องมากกว่าต้องการความเร็วในการทำงาน ผลลัพธ์ที่ได้จึงทำให้การทำงานช้ามาก ใช้สำหรับตรวจสอบความถูกต้องในการทำงานของโปรแกรมเท่านั้น

Pluggable Software Device

ในบางกรณีที่อุปกรณ์ในเครื่องคอมพิวเตอร์ไม่สามารถเร่งความเร็วในการประมวลผลบางอย่างได้ ไดรเรทรีดี จะจำลอง 3D ฮาร์ดแวร์ขึ้นมาโดยใช้ซอฟต์แวร์แต่จะทำให้การทำงานช้าลงไป แม้ว่าจะสามารถใช้คำสั่งพิเศษที่มีอยู่ในซีพียูได้ก็ตาม

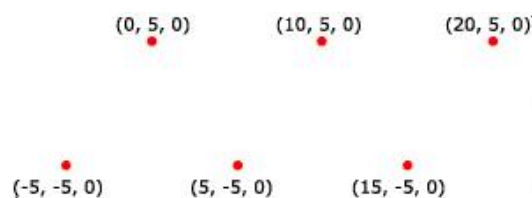
ชนิดของไพรมิทีฟที่สามารถใช้งานได้ (Device-Supported Primitive Types)

ใน ไดรเรทรีดี ดีไวซ์สามารถสร้างและควบคุมไพรมิทีฟ ดังต่อไปนี้ได้

Point List

คือ กลุ่มของเวอร์เท็กซ์ที่จะได้รับการเรนเดอร์เป็นจุดเดี่ยวๆ สามารถนำไปประยุกต์ใช้ในการทำกลุ่มของดวงดาว และเส้นประบนผิวของโพลีกอนได้

ตัวอย่าง จุดที่ได้รับการเรนเดอร์



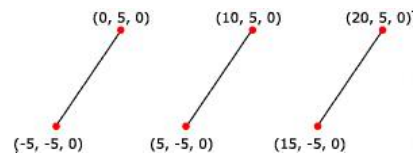
รูปที่ 6-13 รูปแสดง Point List

จุดสามารถมีเมตที่เรียล และเทกเจอร์ได้เช่นกัน โดยสีของเมตที่เรียลหรือเทกเจอร์จะปรากฏเป็นสีของจุดนั้นๆ

Line List

คือ กลุ่มของเวอร์เท็กซ์ที่จะได้รับการเรนเดอร์เป็นเส้นตรงเดี่ยวๆ ซึ่งสามารถนำไปประยุกต์ทำเป็นฝนที่กำลังตกได้

ตัวอย่าง เส้นตรงที่ได้รับการเรนเดอร์



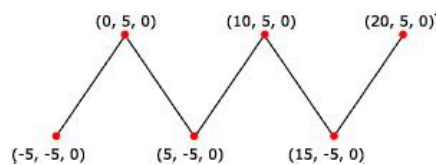
รูปที่ 6-14 รูปแสดง Line List

สามารถกำหนดค่าแมตทรีเรียล และเทกเจอร์ ให้กับเส้นตรงได้เช่นกัน โดยสีจะปรากฏไปตามความยาวของเส้นตรง

Line Strips

คือ เส้นตรงที่มีการเชื่อมต่อปลายเข้าด้วยกัน

ตัวอย่าง การเรนเดอร์ดังรูป

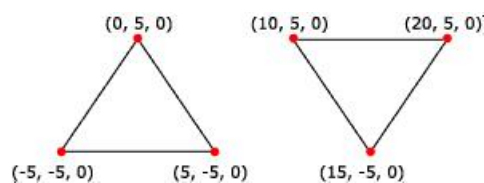


รูปที่ 6-15 รูปแสดง Line Strip

Triangle Lists

คือ จุด 3 จุดที่รวมกันเพื่อสร้างรูป 3 เหลี่ยม 1 รูป โดยไม่เกี่ยวข้องกับรูป 3 เหลี่ยมอื่นๆ เนื่องจากทุกๆ ไพรามิทจะต้องเป็นรูป 3 เหลี่ยม ดังนั้น จำนวนจุดที่ส่งให้กับเรนเดอร์ไปป์ไลน์จะต้องหารด้วย 3 ลงตัว

ตัวอย่าง สามเหลี่ยมที่ได้จากการเรนเดอร์ ดังรูป

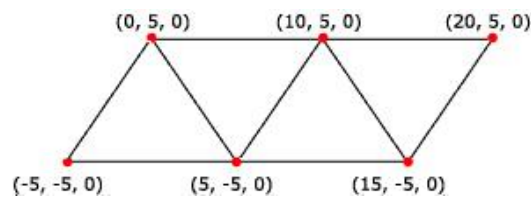


รูปที่ 6-16 รูปแสดง Triangle List

Triangle Strips

คือ กลุ่มของสามเหลี่ยมที่เชื่อมต่อกันไป เนื่องจากการใช้บางจุดร่วมกันจึงทำให้สามารถประหยัดจำนวนข้อมูลได้ จึงทำให้ประสิทธิภาพในการประมวลผลดีขึ้น แต่มีโมเดลจำนวนน้อยที่สามารถทำให้อยู่ในรูปของข้อมูลลักษณะนี้ได้

ตัวอย่าง สามเหลี่ยมที่ได้จากการเรนเดอร์ ดังรูป

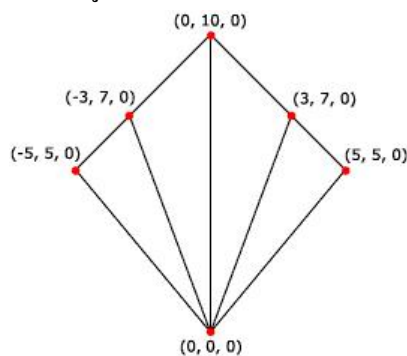


รูปที่ 6-17 รูปแสดง Triangle Strip

Triangle Fans

มีลักษณะคล้ายกับ Triangle strip แต่จะใช้จุดปลายร่วมกัน จึงมีการนำไปใช้งานได้จำกัด

ตัวอย่าง สามเหลี่ยมที่ได้จากการเรนเดอร์ ดังรูป



รูปที่ 6-18 รูปแสดง Triangle Fan

การใช้งานดีไวซ์ (Using Device)

การตรวจสอบฮาร์ดแวร์

ในการตรวจสอบคุณสมบัติที่มีอยู่ในฮาร์ดแวร์ จะต้องใช้เมทอดดังต่อไปนี้

IDirect3D8::CheckDeviceFormat ใช้ในการตรวจสอบว่าฮาร์ดแวร์มีความสามารถในการเรนเดอร์และการเก็บเทกเจอร์บนเซอร์เฟสชนิดใดบ้าง นอกจากนั้นยังใช้ตรวจสอบรูปแบบของสแตนซิลบัฟเฟอร์ (depth-stencil buffer) และเดฟพ์บัฟเฟอร์ (depth buffer) ที่สามารถใช้งานได้

IDirect3D8::CheckDeviceType ใช้ในการตรวจสอบว่าฮาร์ดแวร์มีความสามารถในการเร่งความเร็วในการประมวลผลด้านใดบ้าง, ความสามารถของดีไวซ์ในการทำสวอปเชน (swap chain) ในการแสดงผล และ

ตรวจสอบว่าดีไวซ์มีความสามารถในการเรนเดอร์ในรูปแบบการแสดงผลที่ใช้งานอยู่หรือไม่ (current display format) (จำเป็นในการสร้างโปรแกรมที่ทำงานในวินโดว์)

IDirect3D8::CheckDepthStencilMatch ใช้ตรวจสอบว่ารูปแบบของสแตนซิลบัฟเฟอร์ (depth-stencil buffer format) สามารถใช้งานร่วมกับรูปแบบที่ต้องการจะใช้เรนเดอร์ (render-target format) หรือไม่ ซึ่งก่อนจะใช้เมทอดนี้ควรจะตรวจสอบก่อนว่าฮาร์ดแวร์มีรูปแบบของบัฟเฟอร์แต่ละชนิดอย่างไรบ้าง

การสร้างดีไวซ์

ในการสร้างดีไวซ์ ถ้าเขียนโปรแกรมโดยใช้ภาษา C/C++ จะต้องสร้าง วัตถุใดเรกทรีดีขึ้นมาก่อน หลังจากนั้น จึงใส่ค่าซึ่งกำหนดคุณลักษณะของดีไวซ์ที่ต้องการใช้ข้อมูลโครงสร้าง D3DPRESENT_PARAMETER ดังตัวอย่าง

```
LPDIRECT3DDEVICE8 d3dDevice = NULL;
D3DPRESENT_PARAMETERS d3dpp;
ZeroMemory( &d3dpp, sizeof(d3dpp) );
d3dpp.Windowed = TRUE;
d3dpp.SwapEffect = D3DSWAPEFFECT_COPY_VSYNC;
```

หลักจากนั้นจึงเรียกเมทอด IDirect3D::CreateDevice สำหรับสร้างดีไวซ์ในที่นี้กำหนดให้ใช้โอเพนเตอร์โดยปริยาย (default adapter), สร้างดีไวซ์แบบ HAL และใช้การประมวลผลเวอร์เท็กซ์ด้วยซอฟต์แวร์ (software vertex processing) ดังตัวอย่าง

```
if ( FAILED( g_pD3D->CreateDevice( D3DADAPTER_DEFAULT, D3DDEVTYPE_HAL, hWnd,
                                D3DCREATE_SOFTWARE_VERTEXPROCESSING, &d3dpp, &d3dDevice ) ) )
    return E_FAIL;
```

การกำหนดค่าของเมตริกซ์ทรานสฟอร์มเมชัน (Setting transformation)

ในภาษา C/C++ สามารถเรียกเมทอด IDirect3DDevice::SetTransform โดยส่งค่าเมตริกซ์ที่ต้องการกำหนดและชนิดของทรานสฟอร์มเมชัน ดังตัวอย่างการกำหนดค่าของ view transformation

```
HRESULT hr;
D3DMATRIX view;
// Fill in the view matrix.
if ( FAILED(hr = pDev->SetTransform(D3DTS_VIEW, &view)))
{
    // Code to handle the error goes here;
}
```

ในการกำหนดค่าทรานสฟอร์มเมชันในชนิดอื่นๆ สามารถทำได้โดยกำหนดค่า D3DTS_WORLD, D3DTS_VIEW และ D3DTS_PROJECTION ซึ่งค่าตัวแปรที่สามารถใช้ได้มีการกำหนดเอาไว้ใน D3DTRANSFORMSTATETYPE enumerated type นอกจากนั้นยังประกอบด้วยตัวแปรที่ใช้ในการทรานสฟอร์มเมชันเกี่ยวกับการทำแอนิเมชัน (Animation) และตำแหน่งคู่อันดับของเทกเจอร์ด้วย

6.7 การประมวลผลเวอร์เท็กซ์ (Vertex Processing)

ใน IDirect3DDevice8 อินเทอร์เฟซสนับสนุนการประมวลผลเวอร์เท็กซ์ทั้งในแบบซอฟต์แวร์และฮาร์ดแวร์ ซึ่งโดยปกติแล้วความสามารถของฮาร์ดแวร์ในเครื่องคอมพิวเตอร์แต่ละเครื่องจะแตกต่างกันไป แต่ความสามารถในการประมวลผลของซอฟต์แวร์จะเหมือนกันเสมอ ค่าต่อไปนี้ ใช้ในการควบคุมวิธีการในการประมวลผลเวอร์เท็กซ์สำหรับ Hardware Abstraction Layer (HAL) และ Reference Device

- D3DCREATE_SOFTWARE_VERTEXPROCESSING
- D3DCREATE_HARDWARE_VERTEXPROCESSING
- D3DCREATE_MIXED_VERTEXPROCESSING

D3DCREATE_MIXED_VERTEXPROCESSING คือ การอนุญาตให้ดีไวส์สามารถประมวลผลเวอร์เท็กซ์ได้ทั้งในฮาร์ดแวร์และซอฟต์แวร์

ถ้าใช้ D3DCREATE_HARDWARE_VERTEXPROCESSING จะต้องกำหนดด้วยว่าเป็นการสร้างดีไวส์บริสุทธิ์ (pure device) ด้วยการเพิ่มค่า D3DCREATE_PUREDEVICE เข้าไปด้วย (ใช้โอเพอเรเตอร์ |)

6.8 การเรนเดอร์ (Rendering)

เมทอดในกลุ่มของ DrawPrimitive ใช้ในการเรนเดอร์ภาพจาก 3 มิติ ซึ่งต่อไปนี้จะกล่าวถึงขั้นตอนในการทำงานเพื่อที่จะเรนเดอร์ภาพในแต่ละขั้นตอน

1. การล้างเซอร์เฟส (Clearing Surface)

ก่อนที่จะเรนเดอร์ฉากในขณะหนึ่งๆ จะต้องล้างเซอร์เฟสที่อยู่ในวิวพอร์ต (viewport) ที่เราต้องการเสียก่อนเนื่องจากข้อมูลที่มีอยู่เดิมอาจจะทำให้ภาพที่ปรากฏออกมาไม่มีความคิดพลาดได้ การล้างเซอร์เฟสนอกจากจะเป็นการสั่งให้ล้างข้อมูลบนเซอร์เฟสในบริเวณที่เราต้องการล้างแล้วยังเป็นการเปลี่ยนสถานะของเดฟฟัฟเฟอร์ให้อยู่ในสถานะที่เราต้องการอีกด้วย ซึ่งเราสามารถกำหนดได้ว่าให้ล้างเซอร์เฟสด้วยค่าสีหรือเทกเจอร์ใดๆ

เมทอดที่ใช้ในการล้างเซอร์เฟส คือ IDirect3DDevice8::Clear

หมายเหตุ ในกรณีที่ทราบแน่นอนแล้วว่าการเรนเดอร์ภาพใหม่ทั้งจอภาพก็ไม่มี ความจำเป็นที่จะต้องล้างเซอร์เฟสในส่วน of ค่าสี RGB ซึ่งจะทำให้ได้อัตราเฟรมเพิ่มขึ้นประมาณ 10 เฟรมต่อวินาที

2. การเริ่มต้นและจบฉาก (Beginning and Ending a Scene)

ในภาษา C/C++ ก่อนที่จะทำการเรนเดอร์จะต้องเรียกเมทอด `IDirect3DDevice8::BeginScene` ซึ่ง `BeginScene` เป็นการสั่งให้ระบบทำการตรวจสอบข้อมูลภายในและตรวจสอบว่าเซอร์เฟสที่ต้องการเรนเดอร์มีอยู่จริงหรือไม่ นอกจากนั้นยังกำหนดค่าต่างๆ ภายใน (internal flag) เพื่อกำหนดว่าจะมีการเริ่มการประมวลผลหลักจากเรียกเมทอดนี้แล้วจึงสามารถใช้คำสั่งต่างๆ ในการเรนเดอร์ได้ และการสั่งให้เรนเดอร์โดยไม่เรียกเมทอด `BeginScene` ก่อนจะทำให้การเรนเดอร์ล้มเหลว

หลังจากเรนเดอร์เรียบร้อยแล้วจะต้องเรียกเมทอด `IDirect3DDevice8::EndScene` ซึ่งมีหน้าที่ล้างข้อมูลที่อยู่ในแคช (cache), ตรวจสอบความถูกต้องของเซอร์เฟสที่ทำการเรนเดอร์และทำการล้างค่าต่างๆ ภายใน

นั่นคือ คำสั่งในการเรนเดอร์ทั้งหมดจะต้องอยู่ระหว่างเมทอด `BeginScene` และ `EndScene` เท่านั้น ดังตัวอย่างแสดงการใช้เมทอด

```
HRESULT hr;
if(SUCCEEDED(pDevice->BeginScene()))
{
    // Render primitives only if the scene
    // starts successfully.
    // Close the scene.
    hr = pDevice->EndScene();
    if(FAILED(hr))
        return hr;
}
```

การนำภาพออกมาแสดงผล (Presenting a Scene)

เมทอดเหล่านี้ทำหน้าที่ในการควบคุมสถานะของดีไวซ์ในการแสดงผลบนจอมอนิเตอร์ ประกอบด้วย

- `IDirect3DDevice8::Present`
- `IDirect3DDevice8::Reset`
- `IDirect3DDevice8::GetGammaRamp`
- `IDirect3DDevice8::SetGammaRamp`
- `IDirect3DDevice8::GetRasterStatus`

ซึ่งมีคำหลายคำที่มีความเกี่ยวข้องกับการใช้งานเมทอดเหล่านี้ซึ่งจำเป็นต้องทราบ ดังนี้

- | | |
|--------------|--|
| Front Buffer | คือ ส่วนของเมมโมรีที่จะได้รับการแปลงโดยกราฟิกอแดปเตอร์ (Graphics Adapter) แล้วแสดงผลออกจากหน้าจอมนิเตอร์หรืออุปกรณ์อื่นๆ |
| Back Buffer | คือ เซอร์เฟสที่ข้อมูลจะถูกเลื่อนขึ้นไปเป็น front buffer |

Swap Chain คือ ชุดของ Back Buffer ที่เรียงลำดับเพื่อจะกลายเป็น Front Buffer ซึ่งในการแสดงผลแบบเต็มจอภาพ (full screen) จะมีการแสดงอยู่ในรูปของ flipping DDI (Device Driver Interface) ส่วนการแสดงผลแบบวินโดวส์จะเป็น blitting DDI

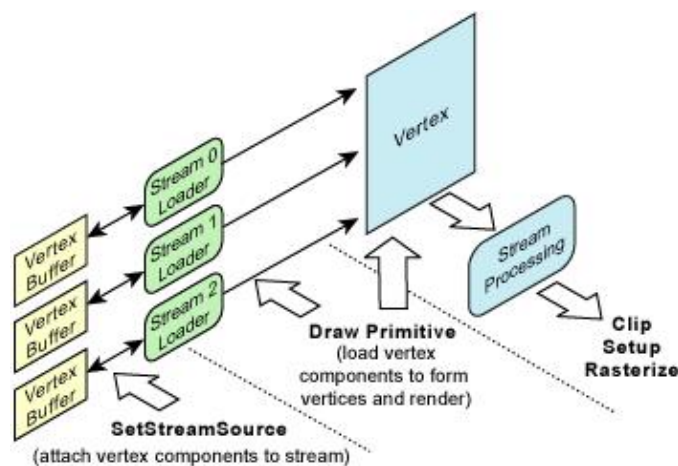
เนื่องจาก Swap Chain เป็นคุณสมบัติของไคลเอนต์ ดังนั้น ดีไวซ์ทุกชนิดจะต้องมีคุณสมบัติในการทำ Swap Chain อย่างน้อย 1 Swap Chain เสมอ นั่นคือ จะต้องมีการมี Back Buffer อย่างน้อย 1 เซอร์เฟส

การเรนเดอร์ไพรมิทีฟ (Rendering Primitives)

- เวอร์เท็กซ์ดาต้าสตรีม (Vertex Data Stream)

ไคลเอนต์ มี API ที่จัดการในส่วนของการเรนเดอร์ไพรมิทีฟ ซึ่งไพรมิทีฟที่จะเรนเดอร์ประกอบขึ้นจากข้อมูลของเวอร์เท็กซ์ซึ่งเก็บอยู่ในดาต้าบัฟเฟอร์ ข้อมูลของเวอร์เท็กซ์อาจจะประกอบด้วยตำแหน่ง, สี, ค่าแกนตั้งฉาก (normal) เป็นต้น

องค์ประกอบของเวอร์เท็กซ์อาจจะประกอบขึ้นจากข้อมูลในเมมโมรีบัฟเฟอร์อันเดียวหรือหลายอันก็ได้ นั่นคือ องค์ประกอบของเวอร์เท็กซ์หนึ่งๆ สามารถแบ่งเก็บในเมมโมรีบัฟเฟอร์มากกว่า 1 บัฟเฟอร์ได้ ดังนั้นในการเรนเดอร์ไพรมิทีฟจะต้องอ่านข้อมูลแล้วนำข้อมูลเหล่านั้นมาประกอบกันจนได้เป็นเวอร์เท็กซ์ที่พร้อมจะประมวลผล ดังแผนภาพ



รูปที่ 6-19 รูปแสดงเวอร์เท็กซ์ดาต้าสตรีม

การเรนเดอร์ไพรมิทีฟประกอบด้วย 2 ขั้นตอน คือ

1. การกำหนดค่าให้กับองค์ประกอบในสตรีม
2. การเรียกเมทอดในกลุ่ม DrawPrimitive เพื่อเรนเดอร์สตรีมเหล่านั้น

การกำหนดแหล่งของสตรีม (Setting the Stream Source)

เมทอด `IDirect3DDevice8::SetStreamSource` ใช้ในการกำหนดเวอร์เท็กซ์บัฟเฟอร์ให้กับสตรีมข้อมูลของดีไวซ์ (device data stream) แต่การสร้างความสัมพันธ์ระหว่างสตรีมข้อมูลเหล่านี้จะยังไม่เกิดขึ้นจนกว่าจะมีการเรียกเมทอดในกลุ่มของ `DrawPrimitive`

สตรีม (stream) คือ อาร์เรย์ของข้อมูลที่เป็นองค์ประกอบต่างๆ แต่ละองค์ประกอบอาจจะประกอบด้วยข้อมูลที่ย่อยลงไปอีกได้ ซึ่งใช้ในการแสดงตำแหน่ง, แกนตั่งฉาก, สี เป็นต้น ค่าพารามิเตอร์ `Stride` ใช้ในการกำหนดขนาดขององค์ประกอบเหล่านี้ มีหน่วยเป็นไบต์ (byte)

การเรนเดอร์โดยใช้เวอร์เท็กซ์และอินเด็กซ์บัฟเฟอร์ (Rendering from Vertex and Index Buffers)

ในการเมทอดที่เกี่ยวข้องกับการวาดโพรมิทิว สามารถแบ่งออกได้ 2 กลุ่ม คือ แบบที่มีอินเด็กซ์ และแบบที่ไม่มีอินเด็กซ์ ซึ่งการอินเด็กซ์ใช้เป็นข้อมูลในการประกอบโพรมิทิวขึ้นมาให้เป็นรูปร่าง ใน ไคเรทรีดีจะเก็บอินเด็กซ์ไว้ในอินเด็กซ์บัฟเฟอร์ โดยสามารถกำหนดอินเด็กซ์บัฟเฟอร์ที่จะใช้งานได้โดยการเรียกเมทอด `IDirect3DDevice8::SetIndices`

เมทอดที่ใช้ในการวาดโพรมิทิว

- `IDirect3DDevice8::DrawPrimitive` ใช้ในการวาดโพรมิทิวที่ไม่มีค่าอินเด็กซ์อยู่ในอินเด็กซ์บัฟเฟอร์
- `IDirect3DDevice8::DrawIndexPrimitive` ใช้ในการวาดโพรมิทิวที่มีค่าอินเด็กซ์อยู่ในอินเด็กซ์บัฟเฟอร์

6.9 สถานะของดีไวซ์ (Device States)

ในไคเรทรีดี ดีไวซ์ทำงานโดยใช้ค่าสถานะในการคำนวณต่างๆ โดยโปรแกรมจะกำหนดค่าสถานะเหล่านี้ ทั้งสถานะที่เกี่ยวข้องกับการคำนวณแสง, การเรนเดอร์ และการควบคุมโมดูลส่วนทรานสฟอร์มเมชัน เป็นต้น สถานะของดีไวซ์สามารถแบ่งออกได้ 2 แบบ ดังนี้

สถานะในการเรนเดอร์ (Render States)

ควบคุมคุณสมบัติในการในส่วนของการลงจุดสี (rasterization module) ซึ่งสามารถใช้ในการเปลี่ยนแปลงคุณสมบัติในการเรนเดอร์, กำหนด shading ที่จะใช้งาน, กำหนดคุณสมบัติของหมอกและกระบวนการในการลงจุดสีอื่นๆ

คุณสมบัติเหล่านี้ในภาษา C/C++ กำหนดได้ด้วยการเรียกเมทอด `IDirect3DDevice8::SetRenderState` ซึ่งค่าสถานะของดีไวซ์ได้ถูกกำหนดเอาไว้แล้วในชนิดข้อมูล `D3DRENDERSTATETYPE`

การประมวลผลเวอร์เท็กซ์แบบฟังก์ชันตายตัว สามารถกำหนดคุณสมบัติการทำงานได้โดยการกำหนดค่าสถานะเรนเดอร์ แต่ถ้าใช้การประมวลผลเวอร์เท็กซ์แบบโปรแกรมได้ (Programmable Vertex Processing) คุณสมบัติหลายๆ อย่างที่กำหนดโดยใช้สถานะในการเรนเดอร์จะไม่มีผล

นอกจากเมทอด `SetRenderState` แล้วยังมีเมทอดอื่นๆ ที่เกี่ยวข้องกับการกำหนดการทำงานของไปป์ไลน์ประมวลผลเวอร์เท็กซ์แบบฟังก์ชันตายตัว ซึ่งเกี่ยวข้องกับการกำหนดค่าเมตริกซ์ทรานสฟอร์มเมชัน, การกำหนดเมตทีเรียล (material) และกำหนดแสง ดังนี้

- `IDirect3DDevice8::SetTransform`
- `IDirect3DDevice8::SetMaterial`
- `IDirect3DDevice8::SetLight`
- `IDirect3DDevice8::LightEnable`

ค่าสถานะของขั้นเทกเจอร์ (Texture Stage States)

สถานะของขั้นเทกเจอร์ ใช้ในการควบคุมรูปแบบของการประมวลผลเทกเจอร์และการกำหนดฟิลเตอร์ (filter) ที่จะใช้งาน

โปรแกรมที่เขียนในภาษา C/C++ สามารถกำหนดคุณสมบัติของเทกเจอร์เหล่านี้ได้โดยการเรียกเมทอด `IDirect3DDevice8::SetTextureStageState` ซึ่งค่าสถานะของขั้นเทกเจอร์ได้ถูกกำหนดเอาไว้แล้วในชนิดข้อมูล `D3DTEXTURESTAGESTATETYPE` และจะต้องใส่เป็นพารามิเตอร์ตัวแรกในเมทอด `SetTextureStageState`

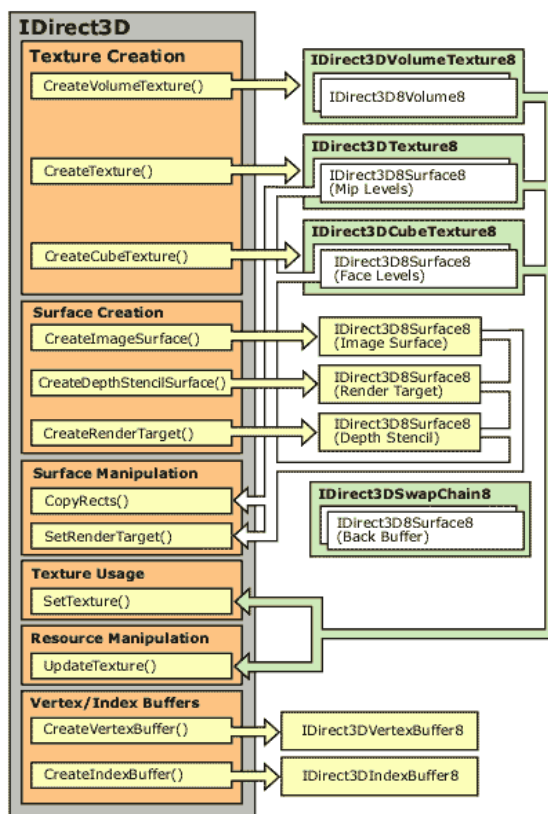
นอกจากนั้น การกำหนดเทกเจอร์เซอร์เฟสที่จะใช้งานสามารถทำได้โดยการเรียกเมทอด `IDirect3DDevice::SetTexture`

6.10 รีซอร์สในไดเรกทรีดี (Direct3D Resources)

ในไดเรกทรีดี รีซอร์ส (resource) คือ เทกเจอร์และบัฟเฟอร์ที่ใช้เก็บข้อมูลที่จะถูกใช้ในการเรนเดอร์ โปรแกรมมีความจำเป็นจะต้องสร้าง, โหลด, คัดลอก และใช้งานรีซอร์สเหล่านี้ ซึ่งประกอบด้วย เทกเจอร์ (texture), เวอร์เท็กซ์บัฟเฟอร์ (Vertex Buffer) และอินเด็กซ์บัฟเฟอร์ (Index Buffer)

ความสัมพันธ์ของรีซอร์ส (Resource Relationships)

แผนภาพต่อไปนี้ แสดงความสัมพันธ์ของเมทอด กับรีซอร์สแต่ละชนิด



รูปที่ 6-20 รูปแสดงความสัมพันธ์ของรีซอร์สในไดเรกทรีดี

ลูกศรที่วิ่งจากซ้ายไปขวาแสดงว่าเมทอดสามารถสร้างรีซอร์สชนิดนั้นได้ ลูกศรที่วิ่งจากทางขวาไปทางซ้ายแสดงว่ารีซอร์สชนิดนั้นสามารถใช้เป็นอาร์กิวเมนต์ (argument) ในเมทอดนั้นได้

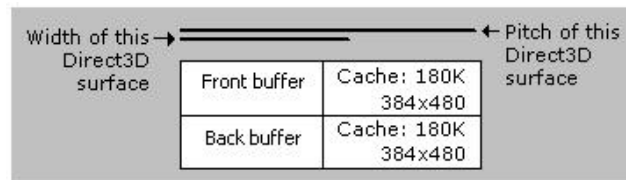
6.11 เซอร์เฟส (Surfaces)

เซอร์เฟส คือ พื้นที่ในเมมโมรีส่วนที่ใช้ในการแสดงผล (display memory) ซึ่งทุกๆ ไปจะอยู่ในเมมโมรีที่อยู่บนการ์ดแสดงผล (display card) แต่ก็สามารถอยู่ในเมมโมรีระบบ (system memory) ได้เช่นกัน เซอร์เฟสถูกบรรจุอยู่ใน IDirect3DSurface8 interface

ความกว้างและค่าพิทช์ (Width and Pitch)

คำว่าความกว้าง (width) และพิทช์ (pitch) มีความหมายแตกต่างกัน ความกว้างในไดเรกทรีดีที่เกี่ยวข้องกับเซอร์เฟสจะหมายถึง มิติของเซอร์เฟสในทางลอจิก (logical) ในส่วนของความกว้าง มีหน่วยเป็นพิกเซล

(pixel) ดังนั้นจึงมีค่าเท่ากันเสมอไม่ว่าพิกเซลจะมีขนาดกี่บิตก็ตาม ส่วนค่าพิทช์ มีความหมายถึงระยะของข้อมูลเป็นจำนวนไบต์ และมักจะมีการรวมเมมโมรีในส่วนที่เป็นแคช (cache) เข้าไปด้วย ดังรูป



รูปที่ 6-21 รูปแสดงความกว้างและค่าพิทช์

ค่าพิทช์มีความจำเป็นในกรณีที่ต้องการเข้าถึงเมมโมรีโดยตรง ซึ่งต้องคำนึงถึงขนาดของเมมโมรีโดยห้ามไปเขียนในส่วนของแคชที่สงวนเอาไว้

รูปแบบของเซอร์เฟส (Surface Format)

รูปแบบของเซอร์เฟสเป็นตัวกำหนดชนิดของข้อมูลในแต่ละพิกเซลซึ่งเก็บอยู่ในเซอร์เฟสเมมโมรี ในไดเรกทรีตี ใช้ชนิดข้อมูล D3DFORMAT ซึ่งเป็นสมาชิกของ D3DSURFACE_DESC ในการอธิบายรูปแบบของเซอร์เฟส โดยสามารถหาค่ารูปแบบของเซอร์เฟสใดๆ ได้โดยเรียกเมทอด IDirect3DSurface8:: GetDesc

6.12 แสง (Lights)

เมื่อมีการกำหนดให้มีการคำนวณแสงในไดเรกทรีตีแล้วในกระบวนการลงจุดสี ซึ่งเป็นขั้นตอนสุดท้ายในเรนเดอร์ไปป์ไลน์ จะมีการคำนวณค่าสีของพิกเซลด้วยผลรวมระหว่างค่าเมตทีเรียล (material), เทกเซล (texel) ซึ่งได้มาจากการปะเทกเจอร์ และความเข้มของแสงและสีของแสงที่ปรากฏอยู่ในฉากซึ่งเกิดจากแหล่งแสงชนิดต่างๆ (light source) หรือระดับของแสงแอมเบียนท์ (ambient light level) เมื่อมีการกำหนดค่าในสถานะการเรนเดอร์ ซึ่งการกำหนดค่าเหล่านี้หมายความว่าต้องการให้ ไดเรกทรีตี จัดการรายละเอียดทั้งหมดให้ แต่สำหรับผู้ที่มีความชำนาญมากๆ ก็สามารถคำนวณค่าเกี่ยวกับแสงได้เองเช่นกัน

การใช้แสงและเมตทีเรียลจะทำให้ภาพที่ได้มีความสวยงามและสมจริงขึ้นอย่างมาก ค่าของเมตทีเรียลเป็นการกำหนดคุณสมบัติในการสะท้อนแสงของพื้นผิว ส่วนค่าของแสงเป็นการกำหนดแสงที่ส่งออกไปเพื่อให้สะท้อน ดังนั้น ถ้าต้องการจะใช้แสงเพื่อเพิ่มความสมจริงและสวยงามให้วัตถุจำเป็นต้องกำหนดค่าเมตทีเรียลให้วัตถุด้วยเสมอ (มิฉะนั้น วัตถุจะอยู่ในสภาพที่มองไม่เห็นหรือเป็นสีดำ)

Direct Light และ Ambient Light

ทั้ง Direct Light และ Ambient Light ต่างก็เป็นวัตถุที่ส่องแสงออกมาได้ แต่วัตถุทั้งสองไม่ขึ้นแก่กันและกัน และให้ผลลัพธ์ในการทำงานที่แตกต่างกัน

Direct Light จะต้องมิติศทางและสีเสมอ ค่าเหล่านี้ถูกใช้ในการคำนวณใน shading algorithm เช่น Gouraud shading ด้วย แสงที่มีชนิดต่างกันจะมีวิธีในการเปล่งแสงแตกต่างกันไป ซึ่งจะทำให้เกิดเอฟเฟกต์ต่างๆ กัน แสงชนิดนี้สามารถกำหนดได้ด้วยการเรียกเมทอด `IDirect3DDevice::SetLight`

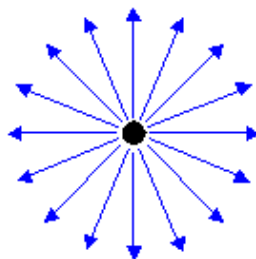
Ambient Light ส่งผลต่อวัตถุทั้งหมดในฉากไม่ว่าอยู่จุดไหนก็ตาม นั่นก็คือ เป็นระดับของแสงที่จะใส่ให้กับวัตถุที่มีอยู่ในฉากทั้งหมด ดังนั้น มันจึงไม่มีทิศทางมีแต่ค่าที่แสดงสีและความเข้มของแสงเท่านั้น สามารถกำหนดค่าได้ด้วยการเรียกเมทอด `IDirect3DDevice::SetRenderState` โดยใส่ค่า `D3DRS_AMBIENT` ในพารามิเตอร์ที่กำหนดชนิดของสถานะ และค่าสีที่ต้องการในพารามิเตอร์ตัวที่สอง

ชนิดของแสง (Light Object)

ในไดเรกทอรีมีแสงอยู่ 3 ชนิดคือ จุดแสง (point light), สปอตไลท์ (spotlight) และแสงทิศทาง (Directional light) ซึ่งแสงแต่ละชนิดมีรายละเอียดดังนี้

จุดแสง (Point Light)

จุดแสงมีสีและตำแหน่งในฉากแต่ไม่ได้มีทิศทางเดียว มันจะให้แสงเท่าๆ กันในทุกทิศทาง ดังรูป

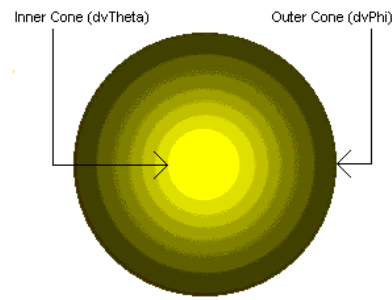


รูปที่ 6-22 รูปแสดงจุดแสง

หลอดไฟฟ้าแบบหลอดไส้เป็นตัวอย่างของจุดแสง ค่าของแสงที่ออกมาจากจุดแสงถูกกำหนดด้วยพารามิเตอร์ attenuation ซึ่งกำหนดลักษณะการจางลงของความเข้มแสง และระยะทางที่แสงส่องไปถึง (range) ไดเรกทอรี ใช้ตำแหน่งของจุดแสง, ระยะทางที่แสงเดินทาง และค่าตั้งฉากของเวกเตอร์เท็กซ์ในการคำนวณความสว่างของพื้นผิว

สปอตไลท์ (Spotlight)

สปอตไลท์ มีทั้งสี, ตำแหน่ง และทิศทาง แสงที่เปล่งออกมาจากสปอตไลท์มีลักษณะเป็นกรวยซึ่งส่วนที่อยู่ในสุดจะมีความสว่างมากที่สุดและค่อยๆ ลดลงเมื่อออกห่างไปเรื่อยๆ ความต่างของความเข้มแสงแสดงดังรูป



รูปที่ 6-23 รูปแสดงสปอตไลท์

ไฟฉายเป็นตัวอย่างของสปอตไลท์ ความเข้มของแสงที่เปล่งออกจากสปอตไลท์ถูกกำหนดด้วยค่า falloff, attenuation และระยะทางที่แสงเดินทางไปถึง (range) เนื่องจากมีพารามิเตอร์จำนวนมากการคำนวณแสงจากสปอตไลท์จึงมีความซับซ้อนมากที่สุดและเปลืองเวลาในการประมวลผลที่สุด

แสงทิศทาง (Directional Light)

มีแต่ค่าของสี และทิศทาง ไม่มีค่าซึ่งแสดงตำแหน่ง เนื่องจากเปล่งแสงในลักษณะคู่ขนาน นั่นคือ เป็นแสงที่ตกกระทบวัตถุทั้งหมดในฉากด้วยทิศทางเดียวกัน เช่นเดียวกับแสงจากดวงอาทิตย์ (เนื่องจากเดินทางมาเป็นระยะทางที่ไกลมากจึงเสมือนว่าเดินทางขนานกันมา) แสงทิศทางไม่มีค่า attenuation หรือระยะทาง ดังนั้นการคำนวณจึงมีความซับซ้อนน้อยที่สุดและเปลืองการประมวลผลน้อยที่สุดเช่นกัน

คุณสมบัติของแสง (Light properties)

คุณสมบัติของแสงอธิบายชนิดของแหล่งแสง (light source) และสี นอกจากนั้นยังกำหนดค่าที่มีความจำเป็นสำหรับแสงในแต่ละชนิด (แสงแต่ละชนิดใช้ค่าพารามิเตอร์ไม่เหมือนกัน) ในไคเรกทรีตีใช้โครงสร้างข้อมูล D3DLIGHT8 ในการเก็บข้อมูลเกี่ยวกับแสงทั้งหมด ซึ่งประกอบด้วยข้อมูลที่แบ่งเป็นกลุ่มต่างๆ ได้ดังนี้

- ชนิดของแสง (Light Type)
- สีของแสง (Light Color)
- ค่าเมตริกซ์รีเลย์ของวัตถุ (Color vertices)
- ตำแหน่ง, ระยะทาง และ attenuation ของแสง
- ทิศทางของแสง (Light direction)
- คุณสมบัติของสปอตไลท์ (spotlight properties)

การใช้แสง (Using Lights)

การกำหนดค่าพารามิเตอร์ของแสง (Setting Light Properties)

ในภาษา C/C++ สร้างกำหนดค่าของแสงได้ด้วยการกำหนดค่าพารามิเตอร์ต่างๆ ที่มีอยู่ในโครงสร้างข้อมูล D3DLIGHT8 แล้วเรียกเมทอด IDirect3DDevice::SetLight ซึ่งจะต้องระบุค่าอินเด็กซ์ของแสงที่จะกำหนดด้วย เนื่องจากในไดเรกทอรีสามารถกำหนดแหล่งแสงได้พร้อมกันสูงสุด 8 แหล่งเท่านั้น

ตัวอย่างการกำหนดค่าของแหล่งแสง

```
D3DLIGHT8 d3dLight;

HRESULT hr;

// Initialize the structure.
ZeroMemory(&d3dLight, sizeof(D3DLIGHT8));

// Set up a white point light.
d3dLight.Type = D3DLIGHT_POINT;
d3dLight.Diffuse.r = 1.0f;
d3dLight.Diffuse.g = 1.0f;
d3dLight.Diffuse.b = 1.0f;
d3dLight.Ambient.r = 1.0f;
d3dLight.Ambient.g = 1.0f;
d3dLight.Ambient.b = 1.0f;
d3dLight.Specular.r = 1.0f;
d3dLight.Specular.g = 1.0f;
d3dLight.Specular.b = 1.0f;
d3dLight.Position.x = 0.0f;
d3dLight.Position.y = 1000.0f;
d3dLight.Position.z = -100.0f;

// Don't attenuate.
d3dLight.Attenuation0 = 1.0f;
d3dLight.dvRange = 1000.0f;

.
```

```
// Set the property information for the first light.
```

```
hr = d3dDevice->SetLight(0, &d3dLight);
```

```
if (FAILED(hr))
```

```
{
```

```
    // Code to handle the error goes here.
```

```
}
```

เมื่อกำหนดค่าของแสงในอินเด็กซ์ใดๆ ไปแล้วสามารถเปลี่ยนแปลงค่าของแสงได้ด้วยการเรียกเมทอด SetLight ด้วยค่าของแสงที่ต้องการอีกครั้ง

การสั่งให้มีการคำนวณแสง (Enabling and Disabling Lights)

เมื่อกำหนดค่าของแหล่งแสงในอินเด็กซ์ใดๆ แล้วการจะให้ไคเรกทรีดี้นำค่าพารามิเตอร์เหล่านั้นไปใช้ในการคำนวณด้วยจะต้องเรียกเมทอด IDirect3DDevice::LightEnable ซึ่งโดยปกติแหล่งแสงจะไม่มีการนำไปใช้คำนวณ (disable) ไว้ เมทอด LightEnable ต้องการอาร์กิวเมนต์ 2 ตัว ค่าแรกคือ อินเด็กซ์ของแหล่งแสง ส่วนค่าที่สองคือ พารามิเตอร์ที่บอกว่าให้นำค่าไปคำนวณหรือไม่ (TRUE ถ้าต้องการให้คำนวณและ FALSE ถ้าไม่ต้องการ)

ตัวอย่างการกำหนดให้ ไคเรกทรีดี นำค่าของแหล่งแสงไปใช้ในการคำนวณ

```
/*
```

```
 * For the purposes of this example, the d3dDevice variable
```

```
 * is a valid pointer to an IDirect3DDevice8 interface.
```

```
*/
```

```
HRESULT hr;
```

```
hr = pd3dDevice->LightEnable(0, TRUE);
```

```
if (FAILED(hr))
```

```
{
```

```
    // Code to handle the error goes here.
```

```
}
```

แต่ถ้ามีการสั่งให้นำค่าในอินเด็กซ์ที่ไม่มีการกำหนดค่าไว้ไปคำนวณ ไคเรกทรีดีจะนำค่าในตารางต่อไปนี้ไปใช้ในการคำนวณแทน

Member	Default
Type	D3DLIGHT_DIRECTIONAL
Diffuse	(R:1, G:1, B:1, A:0)
Specular	(R:0, G:0, B:0, A:0)
Ambient	(R:0, G:0, B:0, A:0)
Position	(0, 0, 0)
Direction	(0, 0, 1)
Range	0
Falloff	0
Attenuation0	0
Attenuation1	0
Attenuation2	0
Theta	0
Phi	0

ตารางที่ 6-1 ตารางแสดงค่าโดยปริยายของแสง

การหาค่าของแสงที่กำหนดไว้แล้ว (Retrieving Light Properties)

สามารถหาค่าคุณสมบัติของแสงที่มีอยู่แล้วได้ด้วยการเรียกเมทอด IDirect3DDevice::GetLight แล้วส่งค่าอาร์กิวเมนต์แรกเป็นอินเด็กซ์ของแสงที่ต้องการ ดังตัวอย่าง

```

HRESULT hr;
D3DLIGHT8 light;

// Get the property information for the first light.
hr = pd3dDevice->GetLight(0, &light);
if (FAILED(hr))
{
    // Code to handle the error goes here.
}

```

ถ้าเรากำหนดค่าอินเด็กซ์ของแสงที่ไม่มีอยู่ในไคลเรทรีดี (มีเฉพาะค่า 0-7 เท่านั้นเนื่องจากมีแสงได้สูงสุดเพียง 8 แหล่งแสง) เมทอด GetLight จะส่งค่า D3DERR_INVALIDCALL คืนมา

6.13 แมตทีเรียล (Materials)

แมตทีเรียลเป็นค่าที่อธิบายวิธีการสะท้อนแสงของโพลีกอนเนื่องจากแหล่งแสงที่มีอยู่ในฉาก มีความสำคัญในการคำนวณค่าดังต่อไปนี้

- วิธีการที่พื้นผิวสะท้อนแสงดิฟฟิวส์ (diffuse light) และแสงแอมเบียน (ambient light)
- ลักษณะความมันวาว (specular highlight)
- โพลีกอนมีคุณสมบัติในการเปล่งแสงหรือไม่ อย่างไร (emissive)

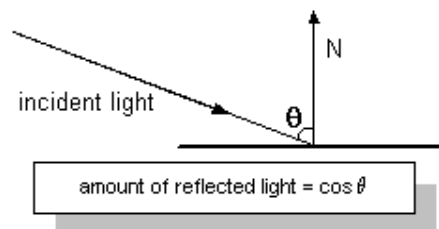
ในภาษา C/C++ คุณสมบัติของแสงใช้โครงสร้างข้อมูล D3DMATERIAL8 ในการเก็บข้อมูลเกี่ยวกับแมตทีเรียล

คุณสมบัติของแมตทีเรียล (Material Properties)

คุณสมบัติของแมตทีเรียลให้รายละเอียดเกี่ยวกับคุณสมบัติต่างๆ ดังต่อไปนี้

1. วิธีการสะท้อนแสงดิฟฟิวส์และแสงแอมเบียน (Diffuse and Ambient Reflection)

ค่า Diffuse และ Ambient เป็นสมาชิกในโครงสร้างข้อมูล D3DMATERIAL8 ซึ่งใช้อธิบายวิธีการสะท้อนแสงทั้งสองชนิด แต่เนื่องจากแสงดิฟฟิวส์โดยปกติแล้วจะส่งผลมากกว่าแสงแอมเบียน คุณสมบัติในการสะท้อนแสงดิฟฟิวส์จึงเป็นคุณสมบัติที่ปรากฏให้เห็นเด่นชัดกว่า เนื่องจากแหล่งแสงส่วนมากจะมีทิศทาง ดังนั้นการสะท้อนแสงดิฟฟิวส์จะสะท้อนได้ดียิ่งขึ้นถ้าทิศทางของแหล่งแสงขนานกับทิศทางตั้งฉากของเวอร์เท็กซ์ แต่ถ้ามุมระหว่างแหล่งแสงกับทิศทางตั้งฉากของเวอร์เท็กซ์เกิดขึ้นมากเท่าไรก็ยิ่งทำให้ความเข้มของแสงอ่อนลงเท่านั้น ปริมาณความเข้มของแสงที่กระทบพื้นผิวคือค่า cosine ของมุมระหว่างทิศทางของแหล่งแสงและทิศทางตั้งฉากของเวอร์เท็กซ์ ดังรูป



รูปที่ 6-24 รูปแสดงวิธีการคำนวณทางแสง

การสะท้อนแสงแอมเบียนก็เหมือนกับแหล่งแสงแอมเบียน คือ ไม่มีทิศทาง การสะท้อนแสงแอมเบียนมีผลต่อวัตถุน้อย แต่มีผลต่อวัตถุทุกวัตถุที่มีอยู่ในฉากและถึงแม้จะมีผลน้อยแต่การเปลี่ยนแปลงของแสงแอมเบียนและค่าการสะท้อนแสงแอมเบียนก็ส่งผลกระทบต่อภาพที่ปรากฏได้เช่นเดียวกัน

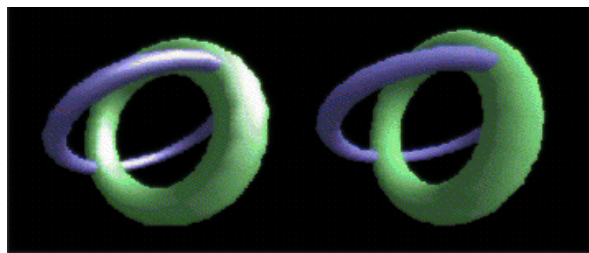
ดังนั้น จึงมีความจำเป็นอย่างยิ่งที่จะต้องรู้คุณสมบัติการสะท้อนแสงทั้งแสงดิฟฟิวส์และแสงแอมเบียนในการกำหนดสีของวัตถุที่อยู่ในฉาก และโดยปกติแล้วจะใช้ค่าเดียวกัน เช่น ถ้าต้องการกำหนดสีให้ผลึกคริสตัลเป็นสีฟ้า ก็จะกำหนดค่าให้คุณสมบัติในการสะท้อนแสงสีฟ้าทั้งในค่าของดิฟฟิวส์และแอมเบียน ดังนั้น ถ้าในฉากมีแหล่งแสงสีขาวก็จะมองเห็นคริสตัลเป็นสีฟ้า แต่ถ้าในฉากมีแต่แหล่งแสงสีแดงก็จะมองเห็นคริสตัลเป็นสีแดง เป็นต้น

การเปล่งแสง (Emission)

ใช้ในการทำให้วัตถุดูเหมือนว่าเปล่งแสงได้ ค่า Emissive ซึ่งเป็นสมาชิกในโครงสร้างข้อมูล D3DMATERIAL8 ใช้ในการกำหนดสีและความโปร่งใสของแสงที่เปล่งออกมาจากวัตถุ เราใช้ค่าการเปล่งแสงของวัตถุในกรณีที่ต้องการทำให้วัตถุดูเหมือนกับเปล่งแสงได้โดยไม่ต้องทำให้ประมวลผลแหล่งแสงเพิ่มเนื่องจากจะใช้การคำนวณมากกว่า แต่สิ่งที่แตกต่างจากการใช้แหล่งแสงคือ แสงที่เปล่งออกมาจะไม่มีผลต่อวัตถุอื่นๆ ในฉากเลย (จึงใช้การคำนวณน้อยกว่า)

ความมันวาว (Specular Reflection)

เป็นค่าที่กำหนดลักษณะความมันวาวบนวัตถุ ซึ่งทำให้วัตถุมีความสว่างยิ่งขึ้น ในโครงสร้างข้อมูล D3DMATERIAL8 ใช้สมาชิก 2 ตัวในการกำหนดลักษณะความมันวาวของวัตถุ คือ Specular ใช้กำหนดสีของความมันวาวที่จะปรากฏ และ Power ใช้กำหนดขนาดของความมันวาว ดังภาพตัวอย่างด้านซ้ายมือ กำหนดค่า Power เท่ากับ 10 และค่า Specular เป็นสีขาว ส่วนด้านขวามือไม่ได้กำหนดค่าความมันวาวให้ (กำหนดให้แต่ค่า Diffuse และค่า Ambient)



รูปที่ 6-25 รูปแสดงสเปกคิวลาร์

การใช้งานแมตทีเรียล (Using Materials)

การกำหนดคุณสมบัติของแมตทีเรียล (Setting Material Properties)

ในภาษา C/C++ การกำหนดคุณสมบัติของแมตทีเรียลสามารถทำได้โดยการกำหนดค่าต่างๆ ในโครงสร้างข้อมูล D3DMATERIAL8 แล้วส่งเป็นอาร์กิวเมนต์ให้กับเมทอด IDirect3DDevice8:: SetMaterial

```
D3DMATERIAL8 mat;
```

```
// Set the RGBA for diffuse reflection.
```

```
mat.Diffuse.r = 0.5f;
```

```
mat.Diffuse.g = 0.0f;
```

```
mat.Diffuse.b = 0.5f;
```

```
mat.Diffuse.a = 1.0f;
```

```
// Set the RGBA for ambient reflection.

mat.Ambient.r = 0.5f;
mat.Ambient.g = 0.0f;
mat.Ambient.b = 0.5f;
mat.Ambient.a = 1.0f;

// Set the color and sharpness of specular highlights.

mat.Specular.r = 1.0f;
mat.Specular.g = 1.0f;
mat.Specular.b = 1.0f;
mat.Specular.a = 1.0f;
mat.Power = 50.0f;
```

หลังจากกำหนดค่าให้กับโครงสร้างข้อมูลแล้วจึงเรียกเมทอด IDirect3DDevice::SetMaterial เพื่อ
กำหนดค่าให้กับดีไวซ์ที่จะทำการเรนเดอร์ไฟรมีทีฟต่อไป ดังตัวอย่าง

```
HRESULT hr;
hr = pd3dDev->SetMaterial(&mat);
if(FAILED(hr))
{
    // Code to handle the error goes here.
}
```

เมื่อมีการสร้างไคลเรทรีดีดีไวซ์ค่าของเมตที่เรียลจะมีการกำหนดโดยอัตโนมัติเป็นค่าดังในตารางต่อไปนี้

Member	Value
Diffuse	(R:1, G:1, B:1, A:0)
Specular	(R:0, G:0, B:0, A:0)
Ambient	(R:0, G:0, B:0, A:0)
Emissive	(R:0, G:0, B:0, A:0)
Power	(0.0)

ตารางที่ 6-2 ค่าโดยปริยายที่กำหนดให้กับเมตที่เรียล

การหาค่าของคุณสมบัติของแมตทีเรียล (Retrieving Material Properties)

สามารถหาค่าคุณสมบัติของแมตทีเรียลที่ใช้งานอยู่ได้โดยการเรียกเมทอด IDirect3DDevice8::

GetMaterial ดังตัวอย่าง

```
HRESULT hr;
```

```
D3DMATERIAL8 mat;
```

```
hr = pd3dDev->GetMaterial(&mat);
```

```
if(FAILED(hr))
```

```
{
```

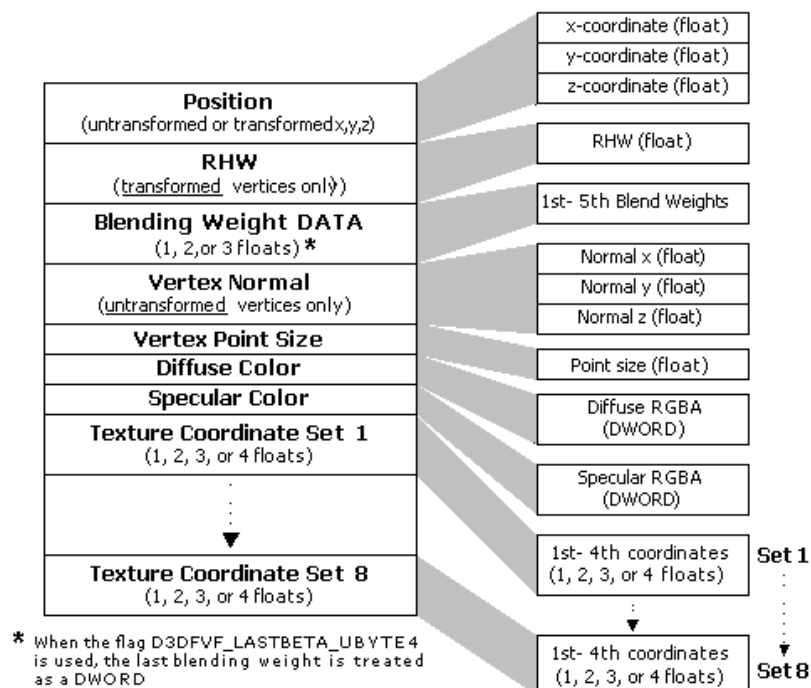
```
    // Code to handle the error goes here.
```

```
}
```

6.14 รูปแบบของเวอร์เท็กซ์ (Vertex Formats)

Flexible Vertex Format (FVF) ใช้ในการอธิบายว่าในข้อมูลเวอร์เท็กซ์มีการเก็บข้อมูลอะไรไว้บ้างซึ่งอยู่ในดาต้าสตรีมเดียวกัน (data stream) ซึ่งโดยปกติแล้วจะใช้ในการกำหนดข้อมูลสำหรับไปป์ไลน์ที่ใช้ประมวลผลเวอร์เท็กซ์แบบฟังก์ชันตายตัว

แผนภาพต่อไปนี้แสดงลำดับของข้อมูลที่สามารถเลือกใช้งานได้ โดยการเก็บข้อมูลจะต้องเก็บเรียงตามลำดับการเก็บในแผนภาพนี้ด้วย



รูปที่ 6-26 รูปแสดงการเรียงลำดับการเก็บข้อมูลในเวอร์เท็กซ์

ค่าคู่อันดับแสดงตำแหน่งของเทกเจอร์ สามารถประกาศใช้ชนิดข้อมูลแบบอื่นก็ได้โดยสามารถมีได้ตั้งแต่ 1 ค่าถึง 4 ค่า

การเขียนโปรแกรมโดยปกติแล้วจะไม่ได้ใช้องค์ประกอบที่มีอยู่ทั้งหมดเนื่องจากบางข้อมูลบางค่าไม่สามารถใช้พร้อมกันได้ เช่น D3DFVF_XYZ และ D3DFVF_XYZRHW

การใช้ indexed vertex blending จะต้องประกาศค่า D3DFVF_LASTBETA_UBYTE4 ในการกำหนดข้อมูลสำหรับการทำแอนิเมชัน (Animation) ด้วย

ตัวอย่างการกำหนดชนิดข้อมูลในเวอร์เทกซ์

```
#define D3DFVF_BLENDVERTEX (D3DFVF_XYZB3|D3DFVF_NORMAL|D3DFVF_TEX1)
```

```
struct BLENDVERTEX
```

```
{
    D3DXVECTOR3 v;           // Referenced as v0 in the vertex shader
    FLOAT    blend1;        // Referenced as v1.x in the vertex shader
    FLOAT    blend2;        // Referenced as v1.y in the vertex shader
    FLOAT    blend3;        // Referenced as v1.z in the vertex shader
                           // v1.w = 1.0 - (v1.x + v1.y + v1.z)
    D3DXVECTOR3 n;          // Referenced as v3 in the vertex shader
    FLOAT    tu, tv;        // Referenced as v7 in the vertex shader
};
```

ตัวอย่างการกำหนดชนิดของข้อมูลในเวอร์เทกซ์โดยใช้ D3DFVF_LAST_UBYTE4 flag

```
#define D3DFVF_BLENDVERTEX (D3DFVF_XYZB4 | D3DFVF_LASTBETA_UBYTE4
```

```
|D3DFVF_NORMAL|D3DFVF_TEX1)
```

```
struct BLENDVERTEX
```

```
{
    D3DXVECTOR3 v;           // Referenced as v0 in the vertex shader
    FLOAT    blend1;        // Referenced as v1.x in the vertex shader
    FLOAT    blend2;        // Referenced as v1.y in the vertex shader
    FLOAT    blend3;        // Referenced as v1.z in the vertex shader
                           // v1.w = 1.0 - (v1.x + v1.y + v1.z)
    DWORD    indices;       // Referenced as v2.xyzw in the vertex shader
    D3DXVECTOR3 n;          // Referenced as v3 in the vertex shader
    FLOAT    tu, tv;        // Referenced as v7 in the vertex shader
};
```

6.15 เทกเจอร์ (Textures)

เทกเจอร์เป็นสิ่งที่ช่วยสร้างความสวยงามและสมจริงให้กับวัตถุใน 3 มิติได้อย่างมาก ดังนั้น ไคเรกทรีดี จึงสนับสนุนคุณสมบัติในการจัดการเทกเจอร์ไว้มากมาย เพื่อให้ผู้ที่พัฒนาสามารถนำคุณสมบัติเหล่านี้ไปใช้ในการประมวลผลเทกเจอร์ในระดับสูงได้

เทกเจอร์ คือ ภาพบิตแมพที่ปะลงบนโพรซีเจอร์ใน ไคเรกทรีดี ซึ่งทำให้ภาพที่ได้มีลักษณะที่เหมือนจริงยิ่งขึ้น เช่น ถ้าเราต้องการจำลองต้นไม้ก็นำภาพบิตแมพที่เป็นลายต้นไม้มาปะในส่วนที่เป็นลำต้น และนำภาพบิตแมพของใบไม้มาปะบริเวณที่เป็นใบไม้ก็จะได้วัตถุที่ดูเหมือนต้นไม้ยิ่งขึ้น

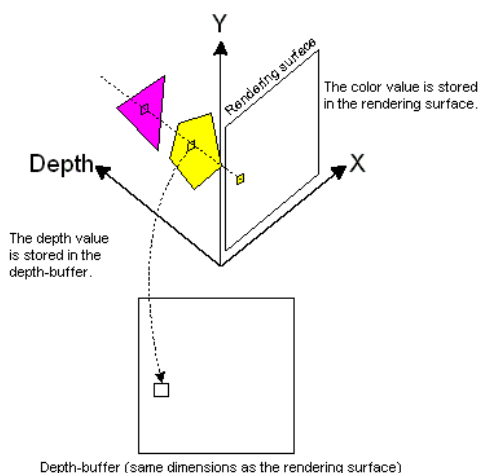
ใน ไคเรกทรีดี เราสามารถกำหนดเทกเจอร์ที่จะใช้ปะบนโพรซีเจอร์ได้ด้วยการเรียกเมทอด `IDirect3DDevice::SetTexture`

เนื่องจากคุณสมบัติในการประมวลผลเทกเจอร์ใน ไคเรกทรีดี มีจำนวนมากและเป็นคุณสมบัติที่มีความซับซ้อนอย่างยิ่ง การใช้งานคุณสมบัติเหล่านั้นจะกล่าวถึงในรายละเอียดเมื่อมีการพัฒนาโปรแกรมในขั้นต่อไป

6.16 เดฟบัฟเฟอร์ (Depth Buffers)

เดฟบัฟเฟอร์ หรือที่รู้จักในชื่อของ Z-Buffer และ W-Buffer คือ คุณสมบัติของฮาร์ดแวร์ในการเก็บข้อมูลเกี่ยวกับความลึกของพิกเซล (pixel) ใน ไคเรกทรีดี ซึ่งจะใช้ในการคำนวณรวมกับข้อมูลที่อยู่ในเรนเดอร์เซอร์เฟสที่ใช้เรนเดอร์ในขณะนั้น (render-target surface) ในการตัดสินใจว่าจุดบนวัตถุใดจะเป็นจุดที่สามารถมองเห็นได้ ในกรณีที่วัตถุมีการซ้อนทับกัน

ในตอนเริ่มต้น ข้อมูลที่เก็บอยู่ในเดฟบัฟเฟอร์จะเป็นค่าที่มากที่สุดที่สามารถเป็นไปได้ และค่าของสีที่กำหนดก็เป็นค่าสีที่กำหนดไว้ตั้งแต่เริ่มต้น (by default) หรือค่าสีที่ใช้เป็นฉากหลัง (background) หรือค่าสีของเทกเจอร์ที่ใช้เป็นฉากหลังในจุดนั้นๆ เมื่อมีการสั่งให้วาดโพลีกอนด้วยคำสั่งใดๆ ก็ตามทุกๆ โพลีกอนจะต้องนำความมาหาจุดตัดในแกน x, y เพื่อนำค่าสีนั้นมาเป็นค่าสีของจุดตัดนั้นและเก็บค่า z ซึ่งแสดงค่าความลึกของจุดไว้ด้วย ในกรณีที่ใช้ Z-Buffer จุดสีที่มีค่าความลึกน้อยกว่าจะได้รับการบันทึกค่าของสีและความลึกเอาไว้ ดังภาพ



รูปที่ 6-27 รูปแสดงวิธีการคำนวณเดฟบัฟเฟอร์

อุปกรณ์ที่มีขายทั่วไปในท้องตลาดส่วนใหญ่จะสนับสนุนการใช้งานเคพพ์เฟอร์ที่เป็น Z-Buffer มากกว่าการใช้งานเคพพ์เฟอร์ที่เป็น W-Buffer ซึ่งเคพพ์เฟอร์ทั้งสองชนิดในบางครั้งก็แสดงภาพที่มีความผิดพลาดได้เช่นกัน

การใช้งานเคพพ์เฟอร์ (Using a Depth Buffer)

เคพพ์เฟอร์เป็นพารามิเตอร์ที่ต้องกำหนดค่าในขณะที่สร้างดีไวซ์ขึ้นมา โดยการกำหนดค่าในโครงสร้างข้อมูล D3DPRESENT_PARAMETERS ดังตัวอย่าง

```
D3DPRESENT_PARAMETERS d3dpp;
ZeroMemory( &d3dpp, sizeof(d3dpp) );
d3dpp.Windowed          = TRUE;
d3dpp.SwapEffect         = D3DSWAPEFFECT_COPY_VSYNC;
d3dpp.EnableAutoDepthStencil = TRUE;
d3dpp.AutoDepthStencilFormat = D3DFMT_D16;
```

หลังจากนั้น จึงเรียกเมทอด IDirect3D::CreateDevice ในการสร้างดีไวซ์และสร้างเคพพ์เฟอร์ ดังตัวอย่าง

```
if ( FAILED( g_pD3D->CreateDevice( D3DADAPTER_DEFAULT, D3DDEVTYPE_HAL, hWnd,
                                D3DCREATE_SOFTWARE_VERTEXPROCESSING, &d3dpp, &d3dDevice ) ) )
    return E_FAIL;
```

การสั่งให้มีการใช้งานเคพพ์เฟอร์ (Enabling Depth Buffering)

หลังจากสร้างเคพพ์เฟอร์แล้วจะต้องสั่งให้มีการใช้งานเคพพ์เฟอร์ด้วยการเรียกเมทอด IDirect3DDevice8::SetRenderState แล้วใส่ค่าอาร์กิวเมนต์ตัวแรกเป็น D3DRS_ZENABLE และใส่ค่าอาร์กิวเมนต์ตัวที่สองเป็น TRUE หรือ D3DZB_TRUE ในกรณีที่ต้องการใช้งาน หรือใส่ค่าเป็น FALSE หรือ D3DZB_FALSE ในกรณีที่ไม่ต้องการใช้งาน (D3DZB_TRUE และ D3DZB_FALSE เป็นสมาชิกในชนิดข้อมูล D3DZBUFFERTYPE)

6.17 เวอร์เท็กซ์บัฟเฟอร์ (Vertex Buffers)

เวอร์เท็กซ์บัฟเฟอร์ คือ เมมโมรีบัฟเฟอร์ที่ใช้เก็บข้อมูลของเวอร์เท็กซ์ และสามารถนำข้อมูลที่มีอยู่ในบัฟเฟอร์นี้ไปเรนเดอร์ได้โดยเรียกเมทอดในกลุ่มของการวาดไพรมิทีฟ (DrawPrimitive) โดยเวอร์เท็กซ์บัฟเฟอร์บรรจุอยู่ใน IDirect3DVertexBuffer8 interface

การสร้างเวอร์เท็กซ์บัฟเฟอร์ (Creating a Vertex Buffer)

เวอร์เท็กซ์บัฟเฟอร์สามารถสร้างขึ้นมาด้วยการเรียกเมทอด `IDirect3DDevice8::CreateVertexBuffer` ซึ่งต้องการพารามิเตอร์ต่างๆ ในการกำหนดคุณสมบัติของเวอร์เท็กซ์บัฟเฟอร์ ดังนี้

- พารามิเตอร์ตัวที่ 1 คือ ขนาดของเวอร์เท็กซ์บัฟเฟอร์ในหน่วยไบต์ซึ่งสามารถคำนวณได้จากการใช้ฟังก์ชัน `sizeof` กับชนิดของข้อมูลที่เก็บข้อมูลเวอร์เท็กซ์
- พารามิเตอร์ตัวที่ 2 คือ ข้อมูลที่ใช้ในการควบคุมการทำงานบางอย่าง เช่น การตัด (clipping) ข้อมูลเวอร์เท็กซ์ที่ไม่ได้อยู่ในวิวพอร์ตหรือไม่ เป็นต้น
- พารามิเตอร์ตัวที่ 3 คือ ค่าที่อธิบายรูปแบบการเก็บข้อมูลในแต่ละเวอร์เท็กซ์ (component of vertex) ซึ่งจะอธิบายในรูปของ FVF (Flexible Vertex Format) โดยการใช้ค่ารวมของ FVF แต่ละชนิด (combination of FVF) หรือใส่ค่าเป็น 0 ในกรณีที่ไม่ใช่ FVF
- พารามิเตอร์ตัวที่ 4 อธิบายบริเวณและคุณสมบัติของเมมโมรี่ที่ใช้เก็บเวอร์เท็กซ์บัฟเฟอร์
- พารามิเตอร์ตัวที่ 5 คือ ตัวแปรที่จะใช้เก็บค่าพอยเตอร์ที่ชี้ไปยังเวอร์เท็กซ์บัฟเฟอร์ที่สร้างขึ้นมา

ตัวอย่างการสร้างเวอร์เท็กซ์บัฟเฟอร์

```
// The custom vertex type
```

```
struct CUSTOMVERTEX {
    FLOAT x, y, z;
    FLOAT rhw;
    DWORD color;
    FLOAT tu, tv; // The texture coordinates
};
```

```
#define D3DFVF_CUSTOMVERTEX (D3DFVF_XYZRHW | D3DFVF_DIFFUSE | D3DFVF_TEX1)
```

```
// Create a clipping-capable vertex buffer. Allocate enough memory in the default memory pool to hold three
// CUSTOMVERTEX structures.
```

```
if ( FAILED( d3dDevice->CreateVertexBuffer(3*sizeof(CUSTOMVERTEX), 0,
D3DFVF_CUSTOMVERTEX, D3DPOOL_DEFAULT, &g_pVB)))
    return E_FAIL;
```

การเข้าถึงข้อมูลที่อยู่ในเวอร์เท็กซ์บัฟเฟอร์ (Accessing the Contents of a Vertex Buffer)

เวอร์เท็กซ์บัฟเฟอร์ให้เราสามารถเข้าถึงเมมโมรี่ที่จองไว้สำหรับข้อมูลเวอร์เท็กซ์ได้โดยตรง ด้วยการใช้อเมทอด `IDirect3DVertexBuffer8::Lock` ในการรับค่าพอยเตอร์ที่ชี้ไปยังเวอร์เท็กซ์บัฟเฟอร์ที่ต้องการจะเข้าถึงข้อมูล ซึ่งการเข้าถึงเมมโมรี่ก็เพื่อที่จะใส่ข้อมูลให้กับเวอร์เท็กซ์บัฟเฟอร์นั่นเองเนื่องจากในขั้นตอนการสร้างยังไม่

ได้มีการกำหนดค่าของข้อมูลภายในเวอร์เท็กซ์บัฟเฟอร์เลย หลังจากที่เราได้กำหนดค่าของข้อมูลในแต่ละเวอร์เท็กซ์ในเวอร์เท็กซ์บัฟเฟอร์แล้ว ต้องเรียกเมทอด IDirect3DVertexBuffer8::Unlock จึงจะนำเวอร์เท็กซ์บัฟเฟอร์ไปใช้ในการเรนเดอร์ได้ ดังตัวอย่าง

```
// To fill the vertex buffer, you need to lock the buffer to
// gain access to the vertices. This mechanism is required because
// vertex buffers may be in device memory.
VOID* pVertices;
if ( FAILED( g_pVB->Lock(      0,                // Fill from the start of the buffer.
                          sizeof(g_Vertices),    // Size of the data to load.
                          (BYTE*)&pVertices, // Returned vertex data.
                          0 ) ) )                // Send default flags to the lock.

    return E_FAIL;

memcpy( pVertices, g_Vertices, sizeof(g_Vertices) );
g_pVB->Unlock();
```

เนื่องจากการใช้ภาษา C/C++ ทำให้สามารถเข้าถึงเมมโมรีได้โดยตรง ซึ่งอาจจะเกิดปัญหาค้างขึ้นในกรณีที่เขียนข้อมูลลงเมมโมรีไม่ถูกต้องซึ่งจะทำให้ข้อมูลที่อยู่ในเวอร์เท็กซ์บัฟเฟอร์ผิดพลาด ดังนั้น ในการเข้าถึงเมมโมรีในแต่ละครั้งจะต้องแน่ใจแล้วว่าข้อมูลที่จะเขียนอยู่ในลำดับและมีขนาดที่ถูกต้อง ถ้าใช้ FVF ข้อมูลที่เป็นองค์ประกอบของเวอร์เท็กซ์จะมีขนาดดังในตารางต่อไปนี้

Vertex Format Flag	Size
D3DFVF_DIFFUSE	sizeof(DWORD)
D3DFVF_NORMAL	sizeof(float) × 3
D3DFVF_SPECULAR	sizeof(DWORD)
D3DFVF_TEXn	sizeof(float) × n × t
D3DFVF_XYZ	sizeof(float) × 3
D3DFVF_XYZRHW	sizeof(float) × 4

ตารางที่ 6-3 ขนาดของข้อมูลที่เป็นองค์ประกอบของเวอร์เท็กซ์

การเรนเดอร์ข้อมูลที่อยู่ในเวอร์เท็กซ์บัฟเฟอร์ (Rendering from a Vertex Buffer)

มีขั้นตอนในการทำงานดังนี้

1. การกำหนดสตรีมที่ใช้เป็นแหล่งของข้อมูลของเวอร์เท็กซ์ สามารถกำหนดได้โดยเรียกเมทอด I ได้
เรกทรีดีDevice8::SetStreamSource ดังตัวอย่าง

```
lpd3dDevice->SetStreamSource( 0, g_pVB, sizeof(CUSTOMVERTEX) );
```

พารามิเตอร์ตัวที่ 1	เป็นการบอกว่าเป็นตัวบอกลำดับของสตรีมมีค่าตั้งแต่ 0 ถึงจำนวนของสตรีมที่มากที่สุด - 1
พารามิเตอร์ตัวที่ 2	คือ เวอร์เท็กซ์บัฟเฟอร์ที่จะใช้งาน
พารามิเตอร์ตัวที่ 3	คือ ขนาดของเวอร์เท็กซ์แต่ละเวอร์เท็กซ์

2. กำหนดเวอร์เท็กซ์เชดเดอร์ (vertex shader) ที่จะใช้งาน สามารถกำหนดได้ด้วยการเรียกเมทอด I ได้
เรกทรีดีDevice::SetVertexShader ดังตัวอย่าง

```
d3dDevice->SetVertexShader( D3DFVF_CUSTOMVERTEX );
```

3. การวาดไพรมิทีฟ (DrawPrimitive) โดยใช้เวอร์เท็กซ์บัฟเฟอร์ สามารถวาดไพรมิทีฟได้โดยการเรียกเมทอด I ได้
เรกทรีดีDevice8::DrawPrimitive ดังตัวอย่าง

```
d3dDevice->DrawPrimitive( D3DPT_TRIANGLELIST, 0, 1 );
```

พารามิเตอร์ตัวที่ 1	คือ ชนิดของไพรมิทีฟที่จะวาด
พารามิเตอร์ตัวที่ 2	คือ ตำแหน่งเริ่มต้นของข้อมูลในเวอร์เท็กซ์บัฟเฟอร์ที่จะนำมาใช้วาด
พารามิเตอร์ตัวที่ 3	คือ จำนวนของไพรมิทีฟที่จะวาด

6.18 อินเด็กซ์บัฟเฟอร์ (Index Buffers)

อินเด็กซ์บัฟเฟอร์ คือ เมมโมรีบัฟเฟอร์ที่ใช้ในการเก็บข้อมูลอินเด็กซ์ (Index data) ซึ่งใช้ในการกำหนดค่าออฟเซต (offset) ในเวอร์เท็กซ์บัฟเฟอร์และใช้ในการเรนเดอร์ไพรมิทีฟ โดยอินเด็กซ์บัฟเฟอร์บรรจุอยู่ใน IDirect3DIndexBuffer8 interface

การสร้างอินเด็กซ์บัฟเฟอร์ (Creating an Index Buffer)

สามารถสร้างอินเด็กซ์บัฟเฟอร์ได้ด้วยการเรียกเมทอด IDirect3DDevice8::CreateIndexBuffer ซึ่งต้องการพารามิเตอร์ดังต่อไปนี้

พารามิเตอร์ตัวที่ 1 ระบุขนาดของอินเด็กซ์บัฟเฟอร์มีหน่วยเป็นไบต์

พารามิเตอร์ตัวที่ 2 ข้อมูลที่ใช้ในการควบคุมการทำงานบางอย่าง เช่น การตัด (clipping) ข้อมูลอินเด็กซ์ที่ไม่ได้อยู่ในวิวพอร์ตหรือไม่ เป็นต้น

พารามิเตอร์ตัวที่ 3 ใช้ระบุขนาดของชนิดข้อมูลของอินเด็กซ์ ซึ่งมี 2 แบบ คือ แบบ 16 บิต จะต้องใช้ค่า D3DFMT_INDEX16 และแบบ 32 บิตจะต้องใช้ค่า D3DFMT_INDEX32 ซึ่งเป็นสมาชิกของชนิดข้อมูล D3DFORMAT enumerated type

พารามิเตอร์ตัวที่ 4 อธิบายบริเวณและคุณสมบัติของเมมโมรีที่ใช้เก็บเวอร์เท็กซ์บัฟเฟอร์

พารามิเตอร์ตัวที่ 5 คือ ตัวแปรที่จะใช้เก็บค่าพอยเตอร์ที่ชี้ไปยังอินเด็กซ์บัฟเฟอร์ที่สร้างขึ้นมา

ตัวอย่างการสร้างอินเด็กซ์เท็กซ์บัฟเฟอร์

```
if( FAILED( d3dDevice->CreateIndexBuffer( 16384 *sizeof(WORD), D3DUSAGE_WRITEONLY,
                                           D3DFMT_INDEX16, D3DPOOL_DEFAULT, &g_IB ) ) )
    return E_FAIL;
```

การเข้าถึงข้อมูลที่ยูนอินเด็กซ์บัฟเฟอร์ (Accessing the Contents of a Index Buffer)

อินเด็กซ์บัฟเฟอร์ให้เราสามารถเข้าถึงเมมโมรีที่จองไว้สำหรับข้อมูลอินเด็กซ์ได้โดยตรง ด้วยการ使用方法 IDirect3DIndexBuffer8::Lock ในการรับค่าพอยเตอร์ที่ชี้ไปยังอินเด็กซ์บัฟเฟอร์ที่ต้องการจะเข้าถึงข้อมูล ซึ่งการเข้าถึงเมมโมรีก็เพื่อที่จะใส่ข้อมูลให้กับอินเด็กซ์บัฟเฟอร์นั่นเองเนื่องจากในขั้นตอนการสร้างยังไม่ได้มีการกำหนดค่าของข้อมูลภายในอินเด็กซ์บัฟเฟอร์เลย หลังจากที่เราได้กำหนดค่าของข้อมูลในแต่ละอินเด็กซ์ในอินเด็กซ์บัฟเฟอร์แล้ว จะต้องเรียกเมทอด IDirect3DIndexBuffer8::Unlock จึงจะนำอินเด็กซ์บัฟเฟอร์ไปใช้ในการเรนเดอร์ได้ ดังตัวอย่าง

```
VOID* pIndices;
if( FAILED( IB->Lock( 0, // Fill from start of the buffer.
                    sizeof(g_Indices), // Size of the data to load.
                    (BYTE**)&pIndices, // Returned index data.
                    0 ) ) ) // Send default flags to the lock.
    return E_FAIL;
memcpy( pIndices, g_Indices, sizeof(g_Vertices) );
IB->Unlock();
```

เช่นเดียวกับ ในกรณีของเวอร์เท็กซ์บัฟเฟอร์ การเข้าถึงเมมโมรีโดยตรงจะต้องระวังเรื่องขนาดของข้อมูลให้ดีเนื่องจากการกำหนดขนาดของข้อมูลผิดจะทำให้ภาพที่แสดงผลออกมาผิดพลาดได้ ซึ่งข้อมูลอินเด็กซ์มี 2 ชนิดเท่านั้น คือ ขนาด 16 บิต และขนาด 32 บิต โดยขนาดของอินเด็กซ์ที่สามารถใช้งานได้สามารถตรวจสอบได้ด้วยการเรียกเมทอด IDirect3DDevice::GetDeviceCaps ซึ่งจะให้รายละเอียดความสามารถของฮาร์ดแวร์แก่เรา ในโครงสร้างข้อมูล D3DCAPS8 ซึ่งถ้าสมาชิกที่ชื่อ MaxVertexIndex มากกว่าค่า 0x0000FFFF จึงจะใช้ข้อมูลอินเด็กซ์ขนาด 32 บิตได้

การเรนเดอร์จากอินเด็กซ์บัฟเฟอร์ (Rendering from an Index Buffer)

มีขั้นตอนในการทำงานดังนี้

1. การกำหนดสตรีมที่ใช้เป็นแหล่งของข้อมูลของเวอร์เท็กซ์ สามารถกำหนดได้โดยเรียกเมทอด IDirect3DDevice8::SetStreamSource ดังตัวอย่าง

```
d3dDevice->SetStreamSource( 0, g_pVB, sizeof(CUSTOMVERTEX) );
```

- | | |
|---------------------|---|
| พารามิเตอร์ตัวที่ 1 | เป็นการบอกว่าเป็นตัวบอกลำดับของสตรีมมีค่าตั้งแต่ 0 ถึงจำนวนของสตรีมที่มากที่สุด - 1 |
| พารามิเตอร์ตัวที่ 2 | คือ เวอร์เท็กซ์บัฟเฟอร์ที่จะใช้งาน |
| พารามิเตอร์ตัวที่ 3 | คือ ขนาดของเวอร์เท็กซ์แต่ละเวอร์เท็กซ์ |

2. กำหนดอินเด็กซ์บัฟเฟอร์ที่จะใช้งานด้วยการเรียกเมทอด IไดเรกทรีดีDevice8::SetIndices ดังตัวอย่าง

```
d3dDevice->SetIndices(IB, 0);
```

- | | |
|---------------------|---|
| พารามิเตอร์ตัวที่ 1 | คือ อินเด็กซ์บัฟเฟอร์ที่จะใช้งาน |
| พารามิเตอร์ตัวที่ 2 | คือ ตำแหน่งของข้อมูลอินเด็กซ์ที่เริ่มใช้งาน |

3. การวาดไพรมิทีฟ (DrawPrimitive) โดยใช้อินเด็กซ์บัฟเฟอร์ สามารถวาดไพรมิทีฟได้โดยการเรียกเมทอด IDirect3D::DrawIndexPrimitive ดังตัวอย่าง

```
d3dDevice->DrawIndexedPrimitive(D3DPT_TRIANGLELIST,0,dwVertices,0,dwIndices /3);
```

- | | |
|---------------------|--|
| พารามิเตอร์ตัวที่ 1 | คือ ชนิดของไพรมิทีฟที่จะวาด |
| พารามิเตอร์ตัวที่ 2 | คือ ค่าต่ำสุดของอินเด็กซ์ที่ใช้ในการเรนเดอร์ |

- พารามิเตอร์ตัวที่ 3 คือ จำนวนของอินเด็กซ์ทั้งหมดที่จะใช้ในการเรนเดอร์ โดยเริ่มนับจากค่า BaseVertexIndex (จากเมทอด SetIndices) + MinIndex (พารามิเตอร์ตัวที่ 2)
- พารามิเตอร์ตัวที่ 4 คือ ตำแหน่งของอินเด็กซ์บัฟเฟอร์
- พารามิเตอร์ตัวที่ 5 คือ จำนวนของไพรมิทีฟที่จะเรนเดอร์

6.19 D3DX Library

นอกจากไลบรารีในส่วนของโคเรกทริตี แล้วไมโครซอฟท์ยังได้พัฒนาไลบรารีเพื่อช่วยในการทำงานเบื้องต้นบางอย่างให้แก่ักพัฒนาอีกด้วย ซึ่งส่วนที่มีการนำมาใช้ในโปรเจกต์นี้คือ ฟังก์ชันที่ทำหน้าที่ในการคำนวณค่าทางคณิตศาสตร์ต่างๆ ซึ่งประกอบด้วยกลุ่มของฟังก์ชัน ดังนี้

ระนาบ (Plane)

ควอเทอร์เนียน (Quaternion)

เวกเตอร์ 2 มิติ (2-D Vector)

เวกเตอร์ 3 มิติ (3-D Vector)

เวกเตอร์ 4 มิติ (4-D Vector)